1. Scrivere un programma che chieda di introdurre 3 numeri A, B e C dalla tastiera e successivamente visualizzi il risultato dei seguenti calcoli:
   1) A–B 2) A–B+C 3) A–B–C
2. Scrivere un programma che chieda due numeri e li memorizzi nelle variabili A e B.
   Il programma deve ora scambiare il contenuto di A e di B (es. se inizialmente A contiene 12 e B 27, dopo lo scambio A contiene 27 e B 12).
   Suggerimento: si considerino 2 bicchieri, uno rosso che contiene acqua e uno blu che contiene aranciata, travasando opportunamente, si vuole che quello rosso contenga aranciata e quello blu acqua.

Soluzioni:

```
/* ================================================================ */

/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>
int main()
{
   int A, B, C, X, Y, Z;

   printf("Primo valore? ");
   scanf("%d", &A);
   printf("Secondo valore? ");
   scanf("%d", &B);
   printf("Terzo valore? ");
   scanf("%d", &C);
   X = A-B;
   printf("Risultato 1 = %d\n", X);
   Y = A-B+C;
   printf("Risultato 2 = %d\n", Y);
   Z = A-B-C;
   printf("Risultato 3 = %d\n", Z);
   return 0;
}


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */
```

```
#include <stdio.h>
int main()
{
   int A, B, Temp;

   printf("Primo valore? ");
   scanf("%d", &A);
   printf("Secondo valore? ");
   scanf("%d", &B);
   Temp = A;
   A = B;
   B = Temp;
   printf("A = ", A);
   printf("B = ", B);
   return 0;
}
```

1. Scrivere un programma che chieda 4 numeri `double` e ne calcoli la media (`double`) con 2 decimali.
2. Scrivere un programma che chieda un valore `double` e lo visualizzi con le 3 specifiche di conversione `%f`, `%e` e `%g`.

Soluzioni:

```
/* ================================================================ */

/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   double a, b, c, d;
   double media;

   printf("Primo valore: ");
   scanf("%lf", &a);
   printf("Secondo valore: ");
   scanf("%lf", &b);
   printf("Terzo valore: ");
   scanf("%lf", &c);
   printf("Quarto valore: ");
   scanf("%lf", &d);
   media = (a+b+c+d)/4;
   printf("La media e': %.2f\n", media);

   return 0;
}


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   double x;

   printf("Introduci un valore: ");
   scanf("%lf", &x);
   printf("Visualizzazione con %%f: %f\n", x);
   printf("Visualizzazione con %%e: %e\n", x);
   printf("Visualizzazione con %%g: %g\n", x);
```

```
    return 0;
}
```

1. Scrivere un programma che chieda 4 numeri `int`, ne calcoli la media, la memorizzi in una variabile `float` e la visualizzi con 2 decimali.
2. Scrivere un programma che chieda un valore `double` di temperatura in gradi Fahrenheit e calcoli i valori delle corrispondenti temperature in gradi Celsius e Kelvin (entrambi con parte frazionaria).

$$C = \frac{5}{9} \cdot (F - 32)$$

$$K = C + 273.15$$

3. Un oggetto che si muove ad una velocità *v* confrontabile con quella della luce *c* ($2.99793 \cdot 10^8$ m/s) si accorcia (nel senso della direzione) e aumenta di massa. Le due grandezze sono (appropriatamente!) modificate da un fattore γ (minore di 1) pari a:

$$\gamma = \sqrt{1 - \left(\frac{v}{c}\right)^2}$$

Si scriva un programma che chieda la lunghezza e la massa di un oggetto fermo e calcoli la variazione delle due grandezze a quella velocità (richiesta in input in km/sec).

4. Si scriva un programma per calcolare la distanza in linea d'aria tra due punti della superficie terrestre, note le coordinate geografiche. Il programma chiede i valori di latitudine (N-S) e di longitudine (E-O) in gradi dei due punti. Per calcolare la distanza si usi la seguente formula (le coordinate Nord e Est sono positive, Sud e Ovest negative). Si ricordi che le funzioni trigonometriche utilizzano i radianti.

$$distanza = \arccos(\ p1+p2+p3\ ) * r$$

dove:
r è il raggio medio della Terra: 6372.795 km
$p1 = \cos(lat1)*\cos(lon1)*\cos(lat2)*\cos(lon2)$
$p2 = \cos(lat1)*\sin(lon1)*\cos(lat2)*\sin(lon2)$
$p3 = \sin(lat1)*\sin(lat2)$
essendo:

lat1	latitudine in gradi del primo punto
lon1	longitudine in gradi del primo punto
lat2	latitudine in gradi del secondo punto
lon2	longitudine in gradi del secondo punto

N.B. La formula considera la terra sferica con raggio medio r: non essendolo, la formula dà un risultato con un errore massimo dello 0.5%.

Per la formula del arco-coseno non si utilizzi la funzione di libreria `acos`, ma la seguente:

$$\arccos(x) = \arctan\left(\frac{-x}{\sqrt{1-x^2}}\right) + \frac{\pi}{2}$$

Calcolare le distanze tra gli aeroporti di:

- Torino (TRN, 45.02° N, 07.65° E)
- Roma (FCO, 41.81° N, 12.25° E)
- Los Angeles (LAX, 33.94° N, 118.40° W)

Nota: $\pi = 4 \cdot \tan^{-1}(1)$

[Risposte: TRN-FCO: 515.20 km, TRN-LAX: 9692.7 km, FCO-LAX: 10205.48 km]

Soluzioni:

```
/* ================================================================ */

/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int a, b, c, d;
   float media;
   printf("Primo valore: ");
   scanf("%d", &a);
   printf("Secondo valore: ");
   scanf("%d", &b);
   printf("Terzo valore: ");
   scanf("%d", &c);
   printf("Quarto valore: ");
   scanf("%d", &d);
   media = (a+b+c+d)/4.0F;
   printf("La media e': %.2f\n", media);

   return 0;
}

/* Considerazioni:
   media = (a+b+c+d)/4.0;
      il 4.0 e' un double, promuove il risultato della parentesi a
      double, l'assegnazione darebbe un warning perche' media e' di
      tipo float
```

```
   media = (float)(a+b+c+d)/4;
      il 4 e' un int, viene promosso a float, corretto
   media = (float)(a+b+c+d)/4.0;
      il risultato della parentesi viene convertito in float, ma il
      4.0 e' un double e questo promuove il risultato della parentesi
      a double, l'assegnazione darebbe un warning perche' media e'
      di tipo float
   media = (float)(a+b+c+d)/4.0F;
      corretto
   media = ((float)a+b+c+d)/4;
      corretto

*/


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   double f, c, k;

   printf("Temperatura in gradi Fahrenheit: ");
   scanf("%lf", &f);

   c = 5.0/9.0 * (f-32);
   k = c + 273.15;
   printf("Celsius: %f\nKelvin: %f\n", c, k);

   return 0;
}

/* Considerazioni:
   c = 5/9 * (f-32);
      da' sempre 0 perche' 5 e 9 sono interi e la divisione tra
      interi da' risultato intero con troncamento (appunto 0)
   c = (f-32) * 5/9;
      corretto, f e' double, promuove il 32, il 5 e il 9 in questo
      ordine in quanto * e / hanno la stessa priorita' e la loro
      associativita' e' da sinistra a destra
*/


/* ================================================================ */
/*   3                                                              */
/* ================================================================ */

#include <stdio.h>
#include <math.h>

int main()
{
   double lung, massa;
   double fattore;
   double v;
   const double c = 2.99793E8; /* metri al secondo */

   printf("Lunghezza da fermo (metri): ");
   scanf("%lf", &lung);
   printf("Massa da fermo (kg): ");
```

```
   scanf("%lf", &massa);
   printf("Velocita' (km/secondo): ");
   scanf("%lf", &v);
   v *= 1000.; /* in metri al secondo */
   fattore = sqrt(1-pow(v/c,2));

   printf("Alla velocita' di: %.0f km/s l'oggetto e' piu' corto di"
          " %f metri e pesa %f kg in piu'\n",
          v/1000., lung-lung*fattore, massa/fattore-massa);

   return 0;
}

/* Nota:
   nell'ultima printf le due stringhe di formato vengono agganciate
   a formarne una sola da parte del compilatore
*/

/* ================================================================ */
/*   4                                                              */
/* ================================================================ */

#include <stdio.h>
#include <math.h>

int main()
{
   double lat1, lon1, lat2, lon2;
   double p1, p2, p3;
   double x, d;
   const double r = 6372.795;
   const double Pi = 4.0 * atan(1.0); /* pi greco */

   printf("Latitudine del punto 1 (gradi): ");
   scanf("%lf", &lat1);
   printf("Longitudine del punto 1 (gradi): ");
   scanf("%lf", &lon1);
   printf("Latitudine del punto 2 (gradi): ");
   scanf("%lf", &lat2);
   printf("Longitudine del punto 2 (gradi): ");
   scanf("%lf", &lon2);

   lat1 = lat1 * Pi / 180.0; /* converte in radianti */
   lon1 = lon1 * Pi / 180.0;
   lat2 = lat2 * Pi / 180.0;
   lon2 = lon2 * Pi / 180.0;

   p1 = cos(lat1) * cos(lon1) * cos(lat2) * cos(lon2);
   p2 = cos(lat1) * sin(lon1) * cos(lat2) * sin(lon2);
   p3 = sin(lat1) * sin(lat2);
   x = p1 + p2 + p3;
   d = ( atan2(-x, sqrt(1 - x * x)) + Pi / 2.0 ) * r;
   printf("Distanza: %f\n", d);

   return 0;
}
```

1. Scrivere un programma che chieda due numeri da tastiera e dei due visualizzi il maggiore (es. se vengono inseriti 12 e 27 visualizza 27).
2. Scrivere un programma che chieda un numero da tastiera e stampi sul video se è pari o dispari (consiglio: calcolare il resto).
3. Scrivere un programma che chieda tre numeri da tastiera e dei tre stampi il maggiore.
4. Scrivere un programma che chieda tre numeri da tastiera e li stampi in ordine decrescente.
5. Si vogliono dividere gli allievi di un corso in tre squadre denominate ROSSA, VERDE e BLU secondo il loro numero di matricola. L'assegnazione avviene con il seguente criterio: l'allievo con matricola 1 va nella squadra ROSSA, quello con matricola 2 nella VERDE, quello con matricola 3 nella BLU, quello con matricola 4 nella ROSSA, quello con 5 nella VERDE ecc.
   Il programma deve chiedere il numero di matricola dell'allievo e indicare a quale squadra è assegnato. Usare il costrutto if.
6. Come il precedente, utilizzare uno `switch`.

7. Scrivere un programma che chieda da tastiera di introdurre un numero intero corrispondente ad un voto e stampi a video
   "`Insufficiente`" se è inferiore a 18,
   "`Appena sufficiente`" (18),
   "`Basso`" (19-20),
   "`Medio`" (21-23),
   "`Buono`" (24-26),
   "`Alto`" (27-29),
   "`Massimo`" (30)
   "`Impossibile`" (tutti gli altri)
   Usare il costrutto `if`.
8. Come il precedente, utilizzare uno `switch`.
9. Si scriva un programma che chieda i tre coefficienti `a`, `b` e `c` di un'equazione di secondo grado e calcoli i valori delle soluzioni se questi sono reali; nel caso non lo siano deve semplicemente scrivere a video "`Valori non reali`".
10. Un anno secolare (divisibile per 100) è bisestile se è divisibile per 400, un anno non secolare è bisestile se è divisibile per 4. Ad esempio l'anno 1900 non era bisestile, il 1996 era bisestile, il 2000 lo era, il 2002 non lo era. Si scriva un programma che chieda all'utente di introdurre l'anno e indichi se è bisestile.

Soluzioni:

```
/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   double a, b;

   printf("Introduci il primo valore: ");
   scanf("%lf", &a);
   printf("Introduci il secondo valore: ");
   scanf("%lf", &b);
   if (a >= b)  /* oppure solo > */
      printf("%f\n", a);
   else
      printf("%f\n", b);

   return 0;
}


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int a;

   printf("Introduci il valore: ");
   scanf("%d", &a);
   if (a % 2 == 0)
      printf("Pari\n");
   else
      printf("Dispari\n");

   return 0;
}


/* Soluzione alternativa */
   if (a / 2 * 2 == 0)


/* ================================================================ */
/*   3                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   double a, b, c;

   printf("Introduci il primo valore: ");
   scanf("%lf", &a);
```

```
   printf("Introduci il secondo valore: ");
   scanf("%lf", &b);
   printf("Introduci il terzo valore: ");
   scanf("%lf", &c);
   if (a > b && a > c) /* verifica se a e' > di entrambi */
      printf("%f\n", a);
   else if (b > c)     /* a non e' il piu' grande, o e' b o e' c */
      printf("%f\n", b);
   else                /* non puo' che essere c */
      printf("%f\n", c);

   return 0;
}


/* ================================================================ */
/*   4                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   double a, b, c;

   printf("Introduci il primo valore: ");
   scanf("%lf", &a);
   printf("Introduci il secondo valore: ");
   scanf("%lf", &b);
   printf("Introduci il terzo valore: ");
   scanf("%lf", &c);
   if (a > b && a > c) /* verifica se a e' > di entrambi */
      if (b > c)
         printf("%f %f %f\n", a, b, c);
      else
         printf("%f %f %f\n", a, c, b);
   else if (b > c)     /* a non e' il piu' grande, o e' b o e' c */
      if (a > c)
         printf("%f %f %f\n", b, a, c);
      else
         printf("%f %f %f\n", b, c, a);
   else                /* non puo' che essere c */
      if (a > b)
         printf("%f %f %f\n", c, a, b);
      else
         printf("%f %f %f\n", c, b, a);

   return 0;
}


/* ================================================================ */
/*   5                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int matricola;

   printf("Introduci il numero di matricola: ");
   scanf("%d", &matricola);
```

```
   if (matricola % 3 == 1)
      printf("ROSSA\n");
   else if (matricola % 3 == 2)
      printf("VERDE\n");
   else
      printf("BLU\n");

   return 0;
}


/* ================================================================ */
/*   6                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int matricola;

   printf("Introduci il numero di matricola: ");
   scanf("%d", &matricola);
   switch (matricola % 3)
   {
   case 1:
      printf("ROSSA\n");
      break;
   case 2:
      printf("VERDE\n");
      break;
   default:
      printf("BLU\n");
      break;
   }

   return 0;
}


/* ================================================================ */
/*   7                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int voto;

   printf("Introduci il voto: ");
   scanf("%d", &voto);
   if (voto <= 0)
      printf("Impossibile\n");
   else if (voto < 18)
      printf("Insufficiente\n");
   else if (voto == 18)
      printf("Appena sufficiente\n");
   else if (voto <= 20)       /* inutile scrivere anche voto > 18 */
      printf("Basso\n");
   else if (voto <= 23)
      printf("Medio\n");
   else if (voto <= 26)
```

```
      printf("Buono\n");
   else if (voto <= 29)
      printf("Alto\n");
   else if (voto == 30)
      printf("Massimo\n");
   else
      printf("Impossibile\n");

   return 0;
}


/* ================================================================ */
/*   8                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int voto;

   printf("Introduci il voto: ");
   scanf("%d", &voto);
   switch (voto)
   {
   case 18:
      printf("Appena sufficiente\n");
      break;
   case 19:
   case 20:
      printf("Basso\n");
      break;
   case 21:
   case 22:
   case 23:
      printf("Medio\n");
      break;
   case 24:
   case 25:
   case 26:
      printf("Buono\n");
      break;
   case 27:
   case 28:
   case 29:
      printf("Alto\n");
      break;
   case 30:
      printf("Massimo\n");
      break;
   default:
      if (voto > 0 && voto < 18)
         printf("Insufficiente\n");
      else
         printf("Impossibile\n");
      break;
   }

   return 0;
}
```

```
/* ================================================================ */
/*   9                                                              */
/* ================================================================ */

#include <stdio.h>
#include <math.h>

int main()
{
   double a, b, c, delta;

   printf("Introduci il valore di a: ");
   scanf("%lf", &a);
   printf("Introduci il valore di b: ");
   scanf("%lf", &b);
   printf("Introduci il valore di c: ");
   scanf("%lf", &c);

   delta = b*b-4*a*c;

   if (delta < 0)
     printf("Soluzioni non reali\n");
   else
   {
      printf("x1 = %f\n", (-b-sqrt(delta))/(2*a)); /* parentesi   */
      printf("x2 = %f\n", (-b+sqrt(delta))/(2*a)); /*  al denomin */
   }

   return 0;
}

/* Nota: la verifica del segno di delta evita di calcolare la
   radice quadrata di un valore negativo */

/* ================================================================ */
/*  10                                                              */
/* ================================================================ */


#include <stdio.h>

int main()
{
   int anno;

   printf("Introduci l'anno: ");
   scanf("%d", &anno);
   if (anno % 400 == 0 || (anno % 100 != 0 && anno % 4 == 0)) /* 1 */
      printf("Bisestile\n");
   else
      printf("Non bisestile\n");

   return 0;
}

/* Note:
   (1) le parentesi sono opzionali
   E' inutile scrivere (anno % 100 == 0 && anno % 400 == 0) al posto
   di (anno % 400 == 0), se e' divisibile per 400 lo e' anche
   per 100 */

/* Soluzione alternativa */
```

```c
#include <stdio.h>

int main()
{
   int anno;
   int d4, d100, d400; /* divisibilita' per 4, 100, 400 */

   printf("Introduci l'anno: ");
   scanf("%d", &anno);
   d4 =   (anno % 4 == 0);    /* parentesi opzionali */
   d100 = (anno % 100 == 0);
   d400 = (anno % 400 == 0);
   if (d400 || !d100 && d4)
      printf("Bisestile\n");
   else
      printf("Non bisestile\n");

   return 0;
}

/* Nota: inutile scrivere (d100 && d400) , come sopra */
```

1. Scrivere un programma che calcoli la media (con parte frazionaria) di 100 valori introdotti dalla tastiera.
2. Scrivere un programma che chieda quanti siano i valori che verranno introdotti dalla tastiera, li chieda tutti e ne stampi la somma e la media.
3. Scrivere un programma che calcoli la media di tutti i valori introdotti dalla tastiera finché non ne viene introdotto uno non compreso tra 18 e 30, ad esempio 9999 (provare proprio questo valore!). La visualizzazione della media deve avvenire solo alla fine (ossia non ogni volta che un valore viene introdotto).
4. Scrivere un programma che richieda N numeri da tastiera e ne calcoli il valore massimo.
5. Scrivere un programma che richieda N numeri da tastiera e ne calcoli il valore massimo, il valore minimo, la somma e la media.
6. Si scriva un programma che calcoli il fattoriale di un numero intero N dato dalla tastiera. Si ricordi che il fattoriale di un numero n (simbolo $n!$) viene calcolato con la seguente formula:

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 2 \cdot 1.$$

7. Scrivere un programma che calcola *i primi N numeri* di Fibonacci, con N introdotto dalla tastiera. I numeri di Fibonacci sono una sequenza di valori interi che inizia con i due valori fissi 1 e 1 e ogni successivo valore è la somma dei due precedenti.
   Ad esempio i primi 10 numeri di Fibonacci sono: 1 1 2 3 5 8 13 21 34 55.
8. Scrivere un programma che calcoli *i primi numeri* di Fibonacci *minori o uguali a N*, con N introdotto dalla tastiera.
   Ad esempio i primi numeri di Fibonacci minori o uguali a 10 sono: 1 1 2 3 5 8.
9. Si scriva un programma per calcolare ex mediante il suo sviluppo in serie:

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots$$

   Ogni frazione aggiunge precisione al risultato, per cui conviene usare valori di n adeguatamente elevati, ad esempio compresi tra 30 e 40. Si verifichi che i risultati calcolati in questo modo siano coerenti con quelli forniti dalla funzione intrinseca `exp` calcolando la differenza dei valori.

10. Si scriva un programma dove il calcolatore determini casualmente un numero intero compreso tra 0 e 99 e chieda all'utente di trovare il numero stesso. Ad ogni input dell'utente il calcolatore risponde con "troppo alto" o "troppo basso", finché non viene trovato il valore corretto. Per generare valori casuali si utilizza la funzione `rand`.

11. Si scriva un programma per calcolare la radice quadrata mediante la formula iterativa di Newton: $$x_{i+1} = \frac{1}{2}\left(x_i + \frac{A}{x_i}\right)$$

Dato il valore A, se ne vuole calcolare la radice quadrata x. La formula data calcola valori di x sempre più precisi.
Inizialmente si considera $x_{i=0} = A$, ricavando un valore x_1 che approssima molto grossolanamente il valore della radice quadrata.
Si riinserisce x_1 nella formula (al posto di x_i) ottenendo un x_2 che è un'approssimazione migliore della precedente.
Si continua in questo modo finché il risultato non varia più (cioè $x_i = x_{i+1}$).

Soluzioni:

```
/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>
#define N 100

int main()
{
   int v, somma;
   int i;

   somma = 0;
   for (i=1; i<=N; i++)
   {
      printf("Inserire il %do valore: ", i);
      scanf("%d", &v);
      somma += v;
   }
   printf("Media: %f\n", (double)somma/N);

   return 0;
}

/* Nota: notare l'uso della #define e il cast di somma */


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int v, somma;
   int i, n;

   somma = 0;
   printf("Quanti valori? ");
   scanf("%d", &n);

   for (i=1; i<=n; i++)
   {
      printf("Inserire il %do valore: ", i);
      scanf("%d", &v);
      somma += v;
   }
   printf("Somma: %d\n", somma);
   printf("Media: %f\n", (double)somma/n);

   return 0;
}


/* ================================================================ */
/*   3                                                              */
/* ================================================================ */

/* Prima soluzione */
```

```c
/* con ciclo while */

#include <stdio.h>

int main()
{
   int v, somma;
   int cont;

   somma = 0;
   cont = 0;
   printf("Inserire il 1o valore: ");
   scanf("%d", &v);
   while (v >= 18 && v <= 30)
   {
      cont++;
      somma += v;
      printf("Inserire il %do valore: ", cont+1);
      scanf("%d", &v);
   }
   if (cont > 0)
      printf("Media: %f\n", (double)somma/cont);

   return 0;
}


/* Altra soluzione */
/* con ciclo do-while */

#include <stdio.h>

int main()
{
   int v, somma;
   int cont;

   somma = 0;
   cont = 0;
   do
   {
      printf("Inserire il %do valore: ", cont+1);
      scanf("%d", &v);
      cont++;
      somma += v;
   }while (v >= 18 && v <= 30);
   cont--;
   somma -= v;
   if (cont > 0)
      printf("Media: %f\n", (double)somma/cont);

   return 0;
}


/* ================================================================ */
/*   4                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
```

```
   int v, max;
   int i, n;

   printf("Quanti? ");
   scanf("%d", &n);

   printf("Inserire il 1o valore: ");  /* chiede il primo valore */
   scanf("%d", &v);
   max = v;             /* il primo valore e' anche l'attuale max */

   for (i=1; i<n; i++)  /* parte da 1 e non da 0 perche' il primo */
   {                    /*  e' gia' stato letto */
      printf("Inserire il %do valore: ", i+1);
      scanf("%d", &v);
      if (v>max)
         max = v;
   }
   printf("Massimo: %d\n", max);

   return 0;
}

/* Nota
Se si imposta Max a 0, il programma funziona correttamente solo sei
valori  dati in input sono tutti positivi; infatti se i valori
immessi fossero tutti negativi, si avrebbe un valore massimo finale
che non e' nemmeno stato dato in input.  Se invece impostiamo il max
al valore del primo numero introdotto (come fatto in questo
esercizio) si e' sicuri che il max abbia un valore tra quelli dati
effettivamente in input. Invece, la soluzione di impostare il max al
valore piu' piccolo possibile per il tipo di dato usato per la
variabile che accetta i dati in input, e' si' accettabile, ma
richiede che si conosca tale valore minimo, inoltre la modifica del
programma per utilizzare un altro tipo di dato richiede la modifica
del valore iniziale di max, cosa che non e' richiesta con il metodo
suggerito.
Sebbene la soluzione di utilizzare il primo valore della sequenza
per inizializzare il valore massimo sia in generale la migliore,
in casi specifici si puo' trovare una soluzione alternativa se
non migliore, ad esempio se il massimo da trovare e' una distanza,
il valore max puo' benissimo essere inizializzato a 0 o a un valore
negativo in quanto non esistono distanze negative e la piu' bassa
e' 0.
*/


/* ================================================================ */
/*   5                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   int v, max, min, somma;
   int i, n;

   printf("Quanti? ");
   scanf("%d", &n);

   printf("Inserire il 1o valore: ");
   scanf("%d", &v);
   max = v;
```

```
   min = v;
   somma = v;

   for (i=1; i<n; i++)
   {
      printf("Inserire il %do valore: ", i+1);
      scanf("%d", &v);
      somma += v;
      if (v>max)
         max = v;
      else if (v<min)  /* nota */
         min = v;
   }
   printf("Massimo: %d\n", max);
   printf("Minimo: %d\n", min);
   printf("Somma: %d\n", somma);
   printf("Media: %f\n", (double)somma/n);

   return 0;
}

/* Nota
Qui si chiedono sia il max sia il min, l'aver selezionato il primo
valore come sia max sia min, permette  di ottimizzare il punto
indicato in quanto lo stesso valore non può essere sia maggiore del
max sia minore del min.
Attenzione invece che, nel caso si fossero inizializzati i valori di
max e min a valori ben definiti (ad esempio a 101 e a -1 se si sa
a priori che i valori sono compresi tra 0 e 100), quell'else NON
puo' essere eliminato in quanto il primo valore letto li deve
cambiare entrambi. Si pensi al caso in cui vi e' un solo valore
in input: eseguendo questo programma con l'else il max verrebbe
il minimo resterebbe quello inizializzato, essendo nel ramo else.
Altro esempio e' quello in cui si hanno numeri sempre crescenti:
viene modificato sempre e solo il valore del max e mai quello del min.
*/


/* ================================================================ */
/*   6                                                              */
/* ================================================================ */

#include<stdio.h>
int main()
{
   int n;
   double fatt;

   printf("Numero di cui calcolare il fattoriale: ");
   scanf("%d", &n);

   fatt = 1;               /* fatt = 1 e' fuori dal for perche' */
   for ( ; n>0 ; n--)      /* non ne e' l'indice (stile) */
      fatt = fatt * n;     /* fatt *= n; */

   printf("Fattoriale: %.0f\n", fatt);

   return 0;
}


/* Altra soluzione */
```

```
   fatt = 1;
   while (n>0)      /* n>1 e' ancora meglio: inutile molt. per 1 */
      fatt *= n--;

/* Altra soluzione */

   fatt = 1;
   for (i=1; i<=n; i++)  /* i=2 e' meglio: inutile moltipl. per 1 */
     fatt *= i;

/* Altra soluzione (peggiore perche' esegue piu' operazioni) */

   fatt = 1;
   for (i=0; i<n; i++)
     fatt = fatt * (n-i);   /* fatt *= n-i; */


/* ================================================================ */
/*   7                                                              */
/* ================================================================ */

#include<stdio.h>

int main()
{
   int i, n;
   double a, b, c;       /* tipi con ampio range */

   printf("Quanti numeri? ");
   scanf("%d", &n);

   a = 1;
   b = 1;
   if (n>=1)
      printf("1 ");
   if (n>=2)
      printf("1 ");
   for (i=2; i<n; i++)   /* n>=3, se n<3 non entra nel for */
   {
      c = a + b;
      printf("%.0f ", c);
      a = b;
      b = c;
   }
   printf("\n");

   return 0;
}

/* Nota
In questo caso si sa a priori quante volte il ciclo debba essere
eseguito, la presenza di una variabile di conteggio suggerisce che
un ciclo FOR sia preferibile agli altri.
*/

/* Altra soluzione */

#include<stdio.h>

int main()
{
   int i, n;
   double a, b, c;       /* tipi con ampio range */
```

```
   printf("Quanti numeri? ");
   scanf("%d", &n);

   a = 0;
   b = 1;
   printf("1 ");
   for (i=1; i<n; i++)
   {
      c = a + b;
      printf("%.0f ", c);
      a = b;
      b = c;
   }
   printf("\n");

   return 0;
}

/* Altra soluzione */

#include<stdio.h>

int main()
{
   int i, n;
   double a, b, c;       /* tipi con ampio range */

   printf("Quanti numeri? ");
   scanf("%d", &n);

   a = 1;
   b = 1;

   for (i=0; i<n; i++)
   {
      c = a + b;
      printf("%.0f ", a);
      a = b;
      b = c;
   }
   printf("\n");

   return 0;
}

/* Altra soluzione */

#include<stdio.h>

int main()
{
   int i, n;
   double a, b;       /* tipi con ampio range */

   printf("Quanti numeri? ");
   scanf("%d", &n);

   a = 0;
   b = 1;
   for (i=0; i<n; i++)
      if (i%2)
      {
```

```
            a += b;
            printf("%.0f ", a);
        }
        else
        {
            b += a;
            printf("%.0f ", b);
        }
    printf("\n");

    return 0;
}



/* ================================================================ */
/*   8                                                              */
/* ================================================================ */

#include<stdio.h>

int main()
{
    double n, a, b, c;     /* tipi con ampio range, anche n */

    printf("Massimo numero? ");
    scanf("%lf", &n);

    a = 1;
    b = 1;
    if (n>=1)
        printf("1 1 ");
    c = a + b;
    while (c<=n)
    {
        printf("%.0f ", c);
        a = b;
        b = c;
        c = a + b;
    }
    printf("\n");

    return 0;
}

/* Nota
In questo caso NON si sa quante volte il ciclo verra' eseguito,
l'assenza di una variabile di conteggio e la presenza di una
condizione sui valori prodotti dal corpo del ciclo suggeriscono che
un ciclo WHILE o DO-WHILE sia preferibile a un ciclo FOR.
*/


/* ================================================================ */
/*   9                                                              */
/* ================================================================ */

#include <stdio.h>
#include <math.h>

int main()
{
    double x, ex;
```

```
   int k;
   int kf; /* fattoriale di k */
   int n;  /* numero di frazioni */

   printf("Numero? ");
   scanf("%lf", &x);
   printf("Quante frazioni? ");
   scanf("%d", &n);

   kf = 1; /* inizializza il fattoriale di k */
   ex = 1.0; /* inizializza la sommatoria con il primo valore */
   for (k=1; k<=n; k++)
   {
      /* calcola k! e lo mette in kf */
      kf *= k;  /* oppure: for(i=k; k>=2; k--) kf *= i; */

      /* calcola e somma la frazione successiva */
      ex += pow(x,k) / kf;
   }

   printf("Esponenziale calcolato: %.20f\n", ex);
   printf("Esponenziale con funzione: %.20f\n", exp(x));
   printf("Differenza: %.20f\n", ex - exp(x));

   return 0;
}

/* Nota: %.20f perche' altrim. mostra solo 6 cifre dopo la virgola */


/* ================================================================ */
/*  10                                                              */
/* ================================================================ */

#include<stdio.h>
#include<time.h>
#include<stdlib.h>
#define MAXVAL 99

int main()
{
   int x, v;

   srand(time(NULL));

   x = rand() % (MAXVAL+1);

   do
   {
      printf("Introduci un valore intero tra 0 e %d: ", MAXVAL);
      scanf("%d", &v);
      if (v<x)
         printf("Troppo basso!\n");
      else if (v>x)
         printf("Troppo alto!\n");
   }while(x!=v);

   printf("Trovato!\n");

   return 0;
}
```

```
/* ================================================================ */
/*  11                                                              */
/* ================================================================ */

#include <stdio.h>

int main()
{
   double a, x, xp;

   printf("Numero? ");
   scanf("%lf", &a);

   if (a == 0)
      printf("%f\n", 0.0);
   else if (a < 0)
      printf("Valore immaginario\n");
   else
   {
      x = a; /* xp e' il valore precedente di x */
      do
      {
         xp = x;                 /* salva la x di prima in xp */
         x = .5 * (xp + a / xp); /* nuova approssimazione x */
      }while (x < xp);           /* continua finche' x varia */
      printf("%.20f\n", x);
   }

   return 0;
}
```

1. Scrivere un programma che chieda quanti valori verranno introdotti dalla tastiera (max 100), li chieda tutti e successivamente li visualizzi dall'ultimo al primo.
2. Scrivere un programma che chieda quanti valori verranno introdotti dalla tastiera (max 100), li chieda tutti e successivamente visualizzi prima tutti i valori pari nell'ordine in cui sono stati inseriti e poi tutti i valori dispari nell'ordine inverso.
   Esempio: dati i valori: 8 1 3 2 8 6 5, il programma visualizza: 8 2 8 6 5 3 1
3. Scrivere un programma che definisca 2 vettori `A` e `B` di uguali dimensioni (la dimensione sia chiesta in input, max 100), chieda in input tutti i valori del primo e successivamente tutti i valori del secondo (devono comparire sul video richieste come le seguenti:
   "`Introdurre il 1° valore di A`",
   "`Introdurre il 2° valore di A`" *ecc.*
   e successivamente
   "`Introdurre il 1° valore di B`",
   "`Introdurre il 2° valore di B`" *ecc.*

Il programma crea un terzo vettore `C` della stessa dimensione di `A` e `B` contenente nel 1° elemento la somma del 1° elemento di `A` e del 1° elemento di `B`, nel 2° elemento la somma del 2° elemento di `A` e del 2° elemento di `B` etc. Alla fine deve visualizzare tutti gli elementi di *posizione* (non indice) pari di `C` (il 2°, il 4°,...) e poi tutti quelli di *posizione* dispari (1°, 3°,...).
Esempio: vettori di lunghezza 4, in `A` sono stati messi i valori: `3 5 2 6` e in `B`: `3 2 6 3`, verranno quindi calcolati e messi in `C` i valori: `6 7 8 9` e quindi stampati i valori: `7 9 6 8`.

4. Scrivere un programma che chieda quanti valori verranno introdotti dalla tastiera (max 100), li chieda tutti e li collochi in un vettore. <u>Successivamente</u>, il programma deve determinare il massimo, il minimo, la somma e la media di questi valori.
5. Scrivere un programma che chieda quanti valori (reali) verranno introdotti dalla tastiera (max 100), li chieda tutti, calcoli e stampi la *media mobile* a 3 valori di questi numeri. Si controlli che il numero di valori da introdurre sia almeno pari a 3.

La media mobile è una media aritmetica su solo una parte dei valori (in questo caso 3), ad esempio se viene data la sequenza di valori:

2.1 4.2 1.3 6.7 3.1 5.5 2.1 4.9 3.0 5.4 3.9

il programma deve calcolare la media di 2.1, 4.2 e 1.3 e stamparla, poi la media di 4.2, 1.3 e 6.7 e stamparla, poi 1.3, 6.7 e 3.1 e stamparla, ecc. fino a 3.0, 5.4 e 3.1:

$\underbrace{2.1 \quad 4.2 \quad 1.3}$ 6.7 3.1 5.5 2.1 4.9 3.0 5.4 3.9

2.1 $\underbrace{4.2 \quad 1.3 \quad 6.7}$ 3.1 5.5 2.1 4.9 3.0 5.4 3.9

2.1 4.2 $\underbrace{1.3 \quad 6.7 \quad 3.1}$ 5.5 2.1 4.9 3.0 5.4 3.9

6. Scrivere un programma che acquisisca da tastiera un vettore di (max 100) valori di tipo intero e identifichi la più lunga sequenza di numeri consecutivi uguali. Se vengono identificate più sequenze della stessa lunghezza, si consideri solo la prima identificata. Il programma deve indicare il valore ripetuto e il numero di ripetizioni di quel valore.
Esempio:
Input: `19 3 15 15 7 9 9 9 9 12 3 3 3`
Output: `numero: 9, ripetizioni: 4`

7. Scrivere un programma che definisca una matrice di valori interi e di dimensioni richieste di volta in volta dall'utente (massimo 10x10) mediante input quali "quante righe?" e "quante colonne?". Successivamente di tutti questi valori determini il massimo, il minimo, la somma e la media.
8. Scrivere un programma che definisca una matrice `MX` di valori interi e di dimensioni richieste di volta in volta dall'utente (massimo 20x26) mediante input quali "quante righe?" e "quante colonne?" e immetta in ciascuna cella un valore casuale (usare **`rand`**) compreso tra 0 e 99. Il programma deve poi visualizzare la matrice con i valori allineati correttamente (`%3d` nella `printf`) e contare quanti valori sono pari e quanti sono dispari. *Si tenga separata la parte relativa allo riempimento della matrice dalle operazioni successive.*

9. Definire la matrice `MX` come nel precedente esercizio, questo programma deve indicare se se **almeno un quarto** dei valori della matrice è pari. Non interessa sapere quanti siano i valori pari, ma solo sapere se ce n'è almeno un certo numero noto a priori (*num_righe* * *num_colonne* / 4, ***attenzione: tronca la parte frazionaria***); quindi quando si arriva a contarne esattamente quel numero si può uscire dai due cicli annidati per non perdere tempo inutilmente (è inutile contare anche gli altri). Per uscire da più cicli annidati si può utilizzare il **`goto`**.
10. Si acquisisca da tastiera una matrice di `int` di dimensioni uguale a `R` righe e `C` colonne (con `R` e `C` costanti predefinite), chieda all'utente le dimensioni `r` e `c` di una matrice tali che `r`≤`R` e `c`≤`C`, visualizzi tutte le sottomatrici di dimensioni `r`×`c` della matrice data la cui somma degli elementi è uguale a zero.
Esempio con R=4, C=5, r=2, c=2:

-2	-2	4	9	7
-9	13	-5	22	8
16	-9	1	-9	2
3	2	33	2	9

Output prodotto:

```
-2 -2
-9 13

13 -5
-9 1
```

11. Scrivere un programma che definisca una matrice 20x26 di valori `int` casuali (0-99), chieda all'utente di introdurre una matrice (dimensioni e valori) e determini se questa seconda è contenuta nella prima, indicando le coordinate di ogni ricorrenza della stessa.

2	12	4	9	7
21	25	9	22	8
16	11	11	9	22
3	2	33	11	9

9	22
11	9

```
Trovata alle coordinate (1,2)
Trovata alle coordinate (2,3)
```

Soluzioni:

```
/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include<stdio.h>
#define MAXNUM 100

int main()
{
   int v[MAXNUM], n, i;

   printf("Quanti valori introdurrai (max %d)? ", MAXNUM);
   scanf("%d", &n);
   if (n>0 && n<=MAXNUM)
   {
      for (i=0; i<n; i++)
      {
         printf("Introduci il %do valore: ", i+1);
         scanf("%d", &v[i]);
      }

      for (i=n-1; i>=0; i--)
         printf("%d\n", v[i]);
   }
   else
      printf("Il valore inserito (%d) non e' corretto\n", n);

   return 0;
}


/* ================================================================ */
```

```
/*   2                                                              */
/* ============================================================== */

#include<stdio.h>
#define MAXNUM 100

int main()
{
   int v[MAXNUM], n, i;

   printf("Quanti valori introdurrai (max %d)? ", MAXNUM);
   scanf("%d", &n);
   if (n>0 && n<=MAXNUM)
   {
      for (i=0; i<n; i++)
      {
         printf("Introduci il %do valore: ", i+1);
         scanf("%d", &v[i]);
      }

      for (i=0; i<n; i++)
         if (v[i] % 2 == 0)
            printf("%d ", v[i]);   /* PARI */
         /* NO ELSE QUI in quanto si vogliono SALTARE i dispari */
      for (i=n-1; i>=0; i--)
         if (v[i] % 2 == 1)
            printf("%d ", v[i]);   /* DISPARI */
         /* NO ELSE QUI in quanto si vogliono SALTARE i pari */
   }
   else
      printf("Il valore inserito (%d) non e' corretto\n", n);

   return 0;
}


/* ============================================================== */
/*   3                                                              */
/* ============================================================== */

#include<stdio.h>
#define MAXNUM 100

int main()
{
   int a[MAXNUM], b[MAXNUM], c[MAXNUM], n, i;

   printf("Quanti valori introdurrai (max %d)? ", MAXNUM);
   scanf("%d", &n);
   if (n>0 && n<=MAXNUM)
   {
      for (i=0; i<n; i++)
      {
         printf("Introduci il %do valore di a: ", i+1);
         scanf("%d", &a[i]);
      }

      for (i=0; i<n; i++)
      {
         printf("Introduci il %do valore di b: ", i+1);
         scanf("%d", &b[i]);
         c[i] = a[i] + b[i];
      }
```

```
      for (i=1; i<n; i+=2)
         printf("%d\n", c[i]);

      for (i=0; i<n; i+=2)
         printf("%d\n", c[i]);
   }
   else
      printf("Il valore inserito (%d) non e' corretto\n", n);

   return 0;
}


/* ================================================================ */
/*   4                                                              */
/* ================================================================ */

#include<stdio.h>
#define MAXNUM 100

int main()
{
   int v[MAXNUM], max, min, somma;
   int n, i;

   do
   {
      printf("Quanti valori introdurrai (max %d)? ", MAXNUM);
      scanf("%d", &n);
   }while(n>MAXNUM || n<=0);   /* continua a chiedere un nuovo
                                  valore per n fintanto che il
                                  valore non e' minore di MAXNUM */

   for (i=0; i<n; i++)
   {
      printf("Introduci il %do valore: ", i+1);
      scanf("%d", &v[i]);
   }

   max = v[0];
   min = v[0];
   somma = v[0];

   for (i=1; i<n; i++)
   {
      somma += v[i];
      if (v[i]>max)
         max = v[i];
      else if (v[i]<min)
         min = v[i];
   }
   printf("Massimo: %d\n", max);
   printf("Minimo: %d\n", min);
   printf("Somma: %d\n", somma);
   printf("Media: %f\n", (double)somma/n);

   return 0;
}


/* ================================================================ */
/*   5                                                              */
```

```
/* ================================================================ */

#include<stdio.h>
#define MAXNUM 100

int main()
{
   double v[MAXNUM];
   int n, i;

   do
   {
      printf("Quanti valori introdurrai (max %d)? ", MAXNUM);
      scanf("%d", &n);
   }while(n>MAXNUM || n<=2);   /* continua a chiedere nuovo val */

   for (i=0; i<n; i++)
   {
      printf("Introduci il %do valore: ", i+1);
      scanf("%lf", &v[i]);
   }

   for (i=0; i<n-2; i++)
      printf("%f ", (v[i]+v[i+1]+v[i+2])/3.0);

   return 0;
}


/* ================================================================ */
/*   6                                                              */
/* ================================================================ */

#include <stdio.h>
#define MAXVALORI 100

int main()
{
   int i, n, quanti, maxQuanti;
   int v[MAXVALORI], valoremax;

   do
   {
      printf("Quanti valori introdurrai (max %d)? ", MAXNUM);
      scanf("%d", &n);
   }while(n>MAXNUM || n<=0);   /* continua a chiedere nuovo val */

   printf("Introduci i valori:\n");
   for (i=0; i<n; i++)
      scanf("%lf", &v[i]);

   maxQuanti=1;    /* e' il primo, quindi il num di ripetiz e' 1 */
   quanti=1;       /* e' il primo */
   valoremax=v[0]; /* e' il primo */
   for (i=1; i<n; i++)
   {
      if (v[i]==v[i-1])  /* se e' uguale al precedente */
      {
         quanti++;
         if (quanti>maxQuanti) /* ricerca del max di quanti */
         {
            maxQuanti=quanti;
            valoremax=v[i];
```

```
         }
      }
      else
         quanti=1;
   }
   printf("numero: %d, ripetizioni: %d\n", valoremax, maxQuanti);

   return 0;
}


/* ================================================================ */
/*   7                                                              */
/* ================================================================ */

#include<stdio.h>
#define MAXRIGHE 10
#define MAXCOLONNE 10

int main()
{
   int Mx[MAXRIGHE][MAXCOLONNE], v, max, min, somma;
   int r, c, i, j;

   /* verifica che r sia un valore appropriato */
   do {
      printf("Quante righe (max %d)? ", MAXRIGHE);
      scanf("%d", &r);
   }while (r>MAXRIGHE || r <= 0);

   /* verifica che c sia un valore appropriato */
   do {
      printf("Quante colonne (max %d)? ", MAXCOLONNE);
      scanf("%d", &c);
   }while (c>MAXCOLONNE || c <= 0);

   for (i=0; i<r; i++)
      for (j=0; j<c; j++)
      {
         printf("Introduci il valore per Mx[%d][%d]: ", i, j);
         scanf("%d", &Mx[i][j]);
      }

   max = Mx[0][0]; /* il primo come max */
   min = Mx[0][0]; /* il primo come min */
   somma = 0;

   for (i=0; i<r; i++)
      for (j=0; j<c; j++)
      {
         v=Mx[i][j];
         somma += v;
         if (v > max)
            max = v;
         else if (v < min)
            min = v;
      }
   printf("Massimo: %d\n", max);
   printf("Minimo: %d\n", min);
   printf("Somma: %d\n", somma);
   printf("Media: %f\n", (double)somma/(r*c));

   return 0;
```

```
}



/* ================================================================ */
/*   8                                                              */
/* ================================================================ */

#include<stdio.h>
#include<time.h>
#include<stdlib.h>
#define MAXRIGHE 20
#define MAXCOLONNE 26
#define MAXCASUALE 99

int main()
{
   int MX[MAXRIGHE][MAXCOLONNE], r, c, i, j, numpari=0;

   srand(time(NULL));

   /* verifica che r sia un valore appropriato */
   do {
      printf("Quante righe (max %d)? ", MAXRIGHE);
      scanf("%d", &r);
   }while (r>MAXRIGHE || r <= 0);

   /* verifica che c sia un valore appropriato */
   do {
      printf("Quante colonne (max %d)? ", MAXCOLONNE);
      scanf("%d", &c);
   }while (c>MAXCOLONNE || c <= 0);

   /* riempie MX di valori casuali */
   for (i=0; i<r; i++)
      for (j=0; j<c; j++)
         MX[i][j] = rand() % (MAXCASUALE+1);

   /* visualizza MX e intanto conta i pari */
   for (i=0; i<r; i++)
   {
      for (j=0; j<c; j++)
      {
         printf("%3d", MX[i][j]);
         if (MX[i][j] % 2 == 0)   /* conta i pari */
            numpari++;
      }
      printf("\n");
   }

   printf("Pari: %d\nDispari: %d\n", numpari, r*c-numpari);

   return 0;
}


/* ================================================================ */
/*   9                                                              */
/* ================================================================ */

#include<stdio.h>
#include<time.h>
#include<stdlib.h>
```

```
#define MAXRIGHE 20
#define MAXCOLONNE 26
#define MAXCASUALE 99

int main()
{
   int MX[MAXRIGHE][MAXCOLONNE], r, c, i, j, numpari=0;
   double unquarto;

   srand(time(NULL));
   do {
      printf("Quante righe (max %d)? ", MAXRIGHE);
      scanf("%d", &r);
   }while (r>MAXRIGHE || r <= 0);
   do {
      printf("Quante colonne (max %d)? ", MAXCOLONNE);
      scanf("%d", &c);
   }while (c>MAXCOLONNE || c <= 0);

   unquarto = r*c/4.0;    /* 4.0 e' double per avere un risultato
                              frazionario */

   /* riempie MX di valori casuali */
   for (i=0; i<r; i++)
      for (j=0; j<c; j++)
         MX[i][j] = rand() % (MAXCASUALE+1);

   for (i=0; i<r; i++)
      for (j=0; j<c; j++)
         if (MX[i][j] % 2 == 0)
         {
            numpari++;
            if (numpari >= unquarto)
               goto fuori;   /* vedi nota */
         }
   fuori:

   if (numpari > unquarto)
      printf("Almeno un quarto dei valori e' pari\n");
   else
      printf("Meno di un quarto dei valori e' pari\n");

   return 0;
}

/* Note
   Il generatore di numeri casuali ha una distribuzione uniforme
   per cui e' estremamente improbabile che meno di 1/4 dei valori
   sia pari.
   Si noti l'uso del goto per uscire immediatamente da due cicli
   annidati. Come scritto nelle slide, l'etichetta deve essere
   collocata subito dopo la chiusura dei cicli piu' esterno da cui
   uscire, cioe' non ci devono essere istruzioni in mezzo che
   verrebbero saltate dal goto. Una soluzione strutturata, ossia
   senza goto, sostituirebbe il ciclo indicato con il seguente:

   esci = NO;
   for (i=0; i<r && esci==NO; i++)
      for (j=0; j<c && esci==NO; j++)
         if (MX[i][j] % 2 == 0)
         {
            numpari++;
            if (numpari > unquarto)
```

```
                esci = SI;
         }

   avendo definito nelle variabili:
   enum {NO, SI} esci;
*/


/* ================================================================ */
/*  10                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#define R 4
#define C 5

int main()
{
   int r, c, i, j, k, l;
   int mx[R][C], somma;

   printf("Introduci i valori della matrice (%d x %d):\n", R, C);
   for (i=0; i<R; i++)
      for (j=0; j<C; j++)
         scanf("%d", &mx[i][j]);

   printf("Dimensioni sottomatrice: ");
   do
   {
      printf("numero righe (max %d): ", R);
      scanf("%d", &r);
   }while (r>R || r <= 0);
   do
   {
      printf("numero colonne (max %d): ", C);
      scanf("%d", &c);
   }while (c>C || c <= 0);

   for (i=0; i<R-r+1; i++)
      for (j=0; j<C-c+1; j++)
      {
         somma=0;
         for (k=0; k<r; k++)
            for (l=0; l<c; l++)
               somma+=mx[i+k][j+l];
         if (somma==0)
         {
            for (k=0; k<r; k++)
            {
               for (l=0; l<c; l++)
                  printf("%d ", mx[i+k][j+l]);
               printf("\n");
            }
            printf("\n");
         }
      }

   return EXIT_SUCCESS;
}

/* Nota
   Per velocizzare, si possono eliminare le somme degli indici di
```

```
   mx[][] nei cicli interni e precalcolare i valori finali dei cicli
   perche' non vengano calcolati ad ogni ciclo:

   ir = R-r+1;    /* valore finale precalcolato di i */
   jc = C-c+1;    /* valore finale precalcolato di j */
   for (i=0; i<ir; i++)
      for (j=0; j<jc; j++)
      {
         somma=0;
         kr = i+r;   /* valore finale precalcolato di k */
         lc = j+c;   /* valore finale precalcolato di l */
         for (k=i; k<kr; k++)
            for (l=j; l<lc; l++)
               somma+=mx[k][l];
         if (somma==0)
         {
            for (k=i; k<kr; k++)
            {
               for (l=j; l<lc; l++)
                  printf("%d ", mx[k][l]);
               printf("\n");
            }
            printf("\n");
         }
      }

   avendo definito:
   int ir, jc, kr, lc;
*/


/* ================================================================ */
/*  11                                                              */
/* ================================================================ */

#include<stdio.h>
#include<time.h>
#include<stdlib.h>
#define MAXRIGHE 20
#define MAXCOLONNE 26
#define MAXCASUALE 99

int main()
{
   int MX[MAXRIGHE][MAXCOLONNE], a[MAXRIGHE][MAXCOLONNE];
   int r, c, ra, ca, i, j;
   int cont;

   srand(time(NULL));

   /* riempie la matrice di valori casuali */
   for (r=0; r<MAXRIGHE; r++)
   {
      for (c=0; c<MAXCOLONNE; c++)
      {
         MX[r][c] = rand() % (MAXCASUALE+1);
         printf("%3d", MX[r][c]); /* li visualizza... */
      }
      printf("\n");
   }

   /* chiede le dimensioni della matrice da cercare */
   do {
```

```
      printf("Quante righe (max %d)? ", MAXRIGHE);
      scanf("%d", &ra);
   }while (ra>MAXRIGHE || ra <= 0);

   do {
      printf("Quante colonne (max %d)? ", MAXCOLONNE);
      scanf("%d", &ca);
   }while (ca>MAXCOLONNE || ca <= 0);

   /* chiede i valori della matrice da cercare */
   for (r=0; r<ra; r++)
      for (c=0; c<ca; c++)
      {
         printf("Elemento a(%d,%d): ", r, c);
         scanf("%d", &a[r][c]);
      }

   /* cerca la matrice a in MX */
   for (r=0; r<=MAXRIGHE-ra; r++)
   {
      for (c=0; c<=MAXCOLONNE-ca; c++)
      {
         cont = 0;
         for (i=0; i<ra; i++)
            for (j=0; j<ca; j++)
               if (MX[r+i][c+j] == a[i][j])
                  cont++;
         if (cont == ra*ca)
            printf("Trovata alle coordinate (%d,%d)\n", r, c);
      }
   }

   return 0;
}

/* Nota
   Il metodo precedente è molto inefficiente, per migliorare
   l'efficienza si interrompe il doppio ciclo piu' interno non
   appena si scopre che un elemento non corrisponde nelle due
   matrici.
   I valori finali dei due for esterni possono essere precalcolati
   per evitare che venga ricalcolato ad ogni ciclo.

   /* cerca la matrice a in MX */
   for (r=0; r<=MAXRIGHE-ra; r++)
   {
      for (c=0; c<=MAXCOLONNE-ca; c++)
      {
         cont = 0;
         for (i=0; i<ra; i++)
            for (j=0; j<ca; j++)
               if (MX[r+i][c+j] != a[i][j])
                  goto next;
         next:

         /* i due cicli interni sono arrivati fino alla fine? */
         if (i == ra)   /* si'*/
            printf("Trovata alle coordinate (%d,%d)\n", r, c);
      }
   }

*/
```

1. Scrivere un programma che date due stringhe in input stampi la **più lunga**. La prima se sono di uguale lunghezza.
2. Scrivere un programma che date due stringhe in input stampi la **maggiore**.
3. Scrivere un programma che chieda in input una stringa e calcoli da quanti caratteri è composta (senza usare la funzione `strlen` ma cercando il carattere `'\0'`)
4. Scrivere un programma che data una stringa in input, la converta tutta in maiuscolo.
5. Scrivere un programma che data una stringa in input verifichi se essa contiene almeno una 'A' tra i primi 10 caratteri.
6. Scrivere un programma che richieda in input una stringa e conti quante cifre essa contiene.
   Esempio
   "Ciao2004! C6?" deve dare 5.
7. Scrivere un programma che richieda in input una stringa e conti di quante lettere maiuscole, lettere minuscole, cifre e altri caratteri è composta
   Esempio
   "Ciao2004! C6?" deve dare:
   maiuscole:2, minuscole: 3, cifre: 5, altri: 3.

8. Scrivere un programma che date in input due stringhe di lunghezza diversa indichi se la più corta è contenuta solo una volta nella più lunga.
9. Scrivere un programma che verifichi se la stringa data in input è palindroma o no ("kayak", "otto", "elle", "anilina").
10. Scrivere un programma che verifichi se la stringa data è composta di due parti uguali, trascurando il carattere centrale se la lunghezza è dispari (es. "CiaoCiao", "CiaoXCiao").
11. Un'immagine è composta da 256x256 punti di colore bianco o nero ed è rappresentata da una matrice di caratteri. Ogni elemento della matrice rappresenta un puntino e vale:

    '1' per rappresentare un punto nero

    '0' per rappresentare un punto bianco.

    Si scriva un programma che codifichi l'immagine sostituendo, per ciascuna riga di punti, le sequenze consecutive dello stesso carattere ('0' o '1') con il carattere stesso seguito dal numero delle sue ripetizioni. Si ripeta codificando le colonne.

Ad es. l'immagine seguente (solo 6x3):

0000000000 *(ci sono 10 zeri)*
0111111100 *(1 zero, 7 uno, 2 zeri)*
0100000100 *(1 zero, 1 uno, 5 zeri, 1 uno, 2 zeri)*

viene codificata per righe come segue:

0 10
0 1 1 7 0 2
0 1 1 1 0 5 1 1 0 2

e per colonne come segue:

0 3
0 1 1 2
0 1 1 1 0 1
. . .

Soluzioni:

```
/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include<stdio.h>
#include<string.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR], b[MAXSTR];

   printf("Stringa 1? ");
   gets(a);
   printf("Stringa 2? ");
   gets(b);
   if (strlen(a) >= strlen(b))
      printf("%s\n", a);
   else
      printf("%s\n", b);

   return 0;
}


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */

#include<stdio.h>
```

```c
#include<string.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR], b[MAXSTR];

   printf("Stringa 1? ");
   gets(a);
   printf("Stringa 2? ");
   gets(b);
   if (strcmp(a,b) >=0 )
      printf("%s\n", a);
   else
      printf("%s\n", b);

   return 0;
}


/* ================================================================ */
/*   3                                                              */
/* ================================================================ */

#include<stdio.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR];
   int cont;

   printf("Stringa? ");
   gets(a);
   cont=0;
   while (a[cont] != '\0')
     cont++;
   printf("%d\n", cont);

   return 0;
}


/* ================================================================ */
/*   4                                                              */
/* ================================================================ */

#include<stdio.h>
#include<string.h>
#include<ctype.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR];
   int i, l;

   printf("Stringa? ");
   gets(a);
   l=(int)strlen(a);
   for (i=0; i<l; i++)
     a[i] = (char)toupper(a[i]);
   printf("%s\n", a);
```

```
   return 0;
}

/* Nota
   La funzione strlen produce un valore intero di tipo size_t,
   poiche' la variabile l e' un int il compilatore potrebbe segnalare
   un Warning per l'assegnazione di un size_t a un int; il cast sul
   valore prodotto dalla strlen serve a eliminare questo warning.
   La funzione toupper produce un valore di tipo int, dovendolo
   assegnare ad
   una variabile di tipo char (a[i]) si avrebbe un warning come nel
   caso precedente; anche qui un cast elimina il warning.
*/


/* ================================================================ */
/*   5                                                              */
/* ================================================================ */

#include<stdio.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR];
   int i, trovato=0;

   printf("Stringa? ");
   gets(a);
   for (i=0; i<10 && a[i] != '\0'; i++) /* (1) */
      if (a[i] == 'A')
         trovato = 1;
   if (trovato == 1)
      printf("Trovato\n");
   else
      printf("Non trovato\n");

   return 0;
}

/* Note
   (1) oppure i<10 && i<strlen(a)

   Errore frequente:
   for (i=0; i<10 && a[i] != '\0'; i++)
      if (a[i] == 'A')
         printf("Trovato\n");
      else
         printf("Non trovato\n");
   Per ciascuno dei caratteri di a[] produce o la scritta "trovato"
   o "Non trovato".
*/


/* Altra soluzione */

#include<stdio.h>
#include<string.h>
#define MAXSTR 80

int main()
{
```

```
   char a[MAXSTR], b[11]={'\0'};
   int i;

   printf("Stringa? ");
   gets(a);
   strncpy(b, a, 10)
   if (strchr(b, 'A') != NULL)
      printf("Trovato\n");
   else
      printf("Non trovato\n");

   return 0;
}

/* Nota: b[] viene inizializzato a tutti \0 perche' la strncpy copia
   solo i caratteri di a[], ma non aggiunge il \0 finale
*/



/* ================================================================ */
/*   6                                                              */
/* ================================================================ */

#include <stdio.h>
#include<string.h>
#include<ctype.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR];
   int i, L, cifre=0;

   printf("Introduci la stringa:");
   gets(a);

   L=(int)strlen(a);
   for (i=0; i<L; i++)
      if (isdigit(a[i]))
         cifre++;

   printf("Cifre: %d\n", cifre);

   return 0;
}


/* ================================================================ */
/*   7                                                              */
/* ================================================================ */

#include <stdio.h>
#include<string.h>
#include<ctype.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR];
   int i, L, maius=0, minus=0, cifre=0;

   printf("Introduci la stringa:");
```

```
   gets(a);

   L=(int)strlen(a);
   for (i=0; i<L; i++)
      if (isupper(a[i]))
         maius++;
      else if (islower(a[i]))
         minus++;
      else if (isdigit(a[i]))
         cifre++;

   printf("Maiuscole: %d\n", maius);
   printf("Minuscole: %d\n", minus);
   printf("Cifre:     %d\n", cifre);
   printf("Altri:     %d\n", L-maius-minus-cifre);

   return 0;
}


/* ================================================================ */
/*   8                                                              */
/* ================================================================ */

#include<stdio.h>
#include<string.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR], b[MAXSTR], c[MAXSTR];
   int i, j, cont=0;
   int la, lb;

   printf("Introduci la prima stringa:");
   gets(a);
   printf("Introduci la seconda stringa:");
   gets(b);
   la = (int)strlen(a);
   lb = (int)strlen(b);
   if (la<lb)
   {
      strcpy(c, a);
      strcpy(a, b);
      strcpy(b, c);
      i = la; la = lb; lb = i;
   }

   for (i=0; i < la-lb+1; i++)
   {
      cont++;
      for (j=0; j<lb; j++)
         if (a[i+j] != b[j])
         {
            cont--;
            break;
         }
   }

   if (cont==1)
      printf("Solo una volta\n");
   else
      printf("Non solo una volta\n");
```

```
   return 0;
}


/* ================================================================ */
/*   9                                                              */
/* ================================================================ */

#include<stdio.h>
#include<string.h>
#include<ctype.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR], b[MAXSTR];
   int i, L;

   printf("Introduci la stringa:");
   gets(a);

   L=(int)strlen(a);
   for (i=0; i<L; i++)
      a[i]=(char)tolower(a[i]);

   for (i=0; i<L; i++)
      b[i]=a[L-1-i];
   b[L]='\0';

   if (strcmp(a, b)==0)
      printf("Palindroma\n");
   else
      printf("Non palindroma\n");


   return 0;
}


/* Altra soluzione */

#include<stdio.h>
#include<string.h>
#include<ctype.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR];
   int i, L, palin;

   printf("Introduci la stringa:");
   gets(a);

   L=(int)strlen(a);
   for (i=0; i<L; i++)
      a[i]=tolower(a[i]);

   palin=1;
   for (i=0; i<L/2; i++)
      if (a[i] != a[L-1-i])
      {
```

```
         palin=0;
         break;
      }

   if (palin)
      printf("Palindroma\n");
   else
      printf("Non palindroma\n");

   return 0;
}

/* Note
   a[i] e' a[0+i], cioe' l'i-esimo carattere a partire da quello di
   posizione 0 verso la fine, a[L-1-i] e' a[(L-1)-i], cioe' l'i-esimo
   carattere a partire dalla posizione L-1 (l'ultima) verso l'inizio.

   Se non si vuole utilizzare la variabile palin per scoprire se si
   e' usciti da ciclo for per effetto del break o per la terminazione
   naturale del ciclo stesso, si puo' veificare il valore di i: se e'
   < L/2 allora e' sicuramente uscito per effetto del break (ma in
   questo caso e' piu' chiaro usare la variabile palin */


/* ================================================================ */
/*   10                                                             */
/* ================================================================ */

#include <stdio.h>
#include <string.h>
#define MAXSTR 80

int main()
{
   char a[MAXSTR];
   int i, L, uguali, dispari;

   printf("Introduci la stringa:");
   gets(a);

   L=(int)strlen(a);
   uguali=1;
   dispari = L % 2;
   for (i=0; i<L/2; i++)
      if (a[i] != a[L/2+i+dispari])
      {
         uguali=0;
         break;
      }

   if (uguali)
      printf("Composta di due parti uguali\n");
   else
      printf("Non composta di due parti uguali\n");

   return 0;
}


/* ================================================================ */
/*   11                                                             */
/* ================================================================ */
```

```
#include <stdio.h>
#include <stdlib.h>
#define NRIGHE 256
#define NCOLONNE 256

int main()
{
   char immagine[NRIGHE][NCOLONNE+1]={0}; /* il tipo intero piu'  piccolo */
   int r, c, conta0, conta1;
   enum {ZERO, UNO} stato;

   for (r=0; r<NRIGHE; r++)
      gets(immagine[r]);

   /* compressione per righe */
   conta0=conta1=0;
   for (r=0; r<NRIGHE; r++)
   {
      if (immagine[r][0] == '0') /* stato iniziale */
      {
         stato=ZERO;
         conta0=1;
      }
      else
      {
         stato=UNO;
         conta1=1;
      }
      for (c=1; c<NCOLONNE; c++)
      {
         if (stato==ZERO)
            if (immagine[r][c]=='0')
               conta0++;
            else
            {
               printf("0 %d ", conta0);
               stato=UNO;
               conta1=1;
            }
         else /* if (stato==UNO) */
            if (immagine[r][c]=='1')
               conta1++;
            else
            {
               printf("1 %d ", conta1);
               stato=ZERO;
               conta0=1;
            }
      }
      if (stato==ZERO)
         printf("0 %d\n", conta0);
      else
         printf("1 %d\n", conta1);
   }

   /* compressione per colonne */
   conta0=conta1=0;
   for (c=0; c<NCOLONNE; c++)
   {
      if (immagine[0][c] == '0') /* stato iniziale */
      {
         stato=ZERO;
         conta0=1;
```

```
      }
      else
      {
         stato=UNO;
         conta1=1;
      }
      for (r=1; r<NRIGHE; r++)
      {
         if (stato==ZERO)
            if (immagine[r][c]=='0')
               conta0++;
            else
            {
               printf("0 %d ", conta0);
               stato=UNO;
               conta1=1;
            }
         else if (stato==UNO)
            if (immagine[r][c]=='1')
               conta1++;
            else
            {
               printf("1 %d ", conta1);
               stato=ZERO;
               conta0=1;
            }
      }

      /* manda in output la codifica dell'ultima sequenza */
      if (stato==ZERO)
         printf("0 %d\n", conta0);
      else
         printf("1 %d\n", conta1);
   }

   return EXIT_SUCCESS;
}

/* ================================================================ */

/* Nota
   L'output avviene solo durante le transizioni da uno stato
   all'altro quindi bisogna esplicitamente mandare in output anche
   l'ultima sequenza che non essendo seguita da una transizione non
   viene altrimenti emessa
*/
```

1. `int** x;`
   `x` è una variabile di tipo puntatore
   a un puntatore variabile
   a un oggetto variabile di tipo `int`
2. `const int** x;`
   `x` è una variabile di tipo puntatore
   a un puntatore variabile
   a un oggetto costante di tipo `int`
3. `int ** const x;`
   `x` è una costante di tipo puntatore
   a un puntatore variabile
   a un oggetto variabile di tipo `int`

4. `int * const * x`
   `x` è una variabile di tipo puntatore
   a un puntatore costante
   a un oggetto variabile di tipo `int`

5. `const int * const * x`
   `x` è una variabile di tipo puntatore
   a un puntatore costante
   a un oggetto costante di tipo `int`

6. `const int * const * const x`
   `x` è una costante di tipo puntatore
   a un puntatore costante
   a un oggetto costante di tipo `int`

Soluzioni:

```
/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#define MAXNUM 100

int main()
{
   char v[MAXNUM], *p;

   printf("Introduci la stringa: ");
   gets(v);

   p=v;
   while (*p != '\0')
      p++;

   printf("Lunghezza: %d\n", p-v);

   return EXIT_SUCCESS;
}


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100

int main()
{
   char v[MAX], *p, *q;
   int palin=1;

   printf("Introduci la stringa: ");
   gets(v);

   p=v;             /* punta al primo carattere */
   q=v+strlen(v)-1; /* punta all'ultimo carattere */
   while (p < q)
   {
      if (*p++ != *q--)
      {
         palin = 0;
         break;
      }
   }

   if (palin)
      printf("Palindroma\n");
   else
      printf("Non palindroma\n");

   return EXIT_SUCCESS;
}

/* Altra soluzione */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 100

int main()
{
   char v[MAX], *p, *q;

   printf("Introduci la stringa: ");
   gets(v);

   p=v;             /* punta al primo carattere */
   q=v+strlen(v)-1; /* punta all'ultimo carattere */
   while (p < q  &&  *p++ == *q--)
     ;
   if (*--p == *++q) /* se si e' usciti dal while non perche' i
                        precedenti due car erano diversi allora P */
      printf("Palindroma\n");
   else
      printf("Non palindroma\n");

   return EXIT_SUCCESS;
}
/* Nota
   Nella if si verifica se si e' usciti dal while 1) perche' i due
   caratteri puntati da p e q erano diversi, oppure 2) perche' sono
   state verificate tutte le coppie di caratteri: in questo caso i
```

```
   due ultimi caratteri erano uguali
*/


/* ================================================================ */
/*   3                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#define MAXNUM 100

int main()
{
   int a[MAXNUM], t;
   int i, j, n;

   printf("Quanti valori? ");
   scanf("%d", &n);

   for (i=0; i<n; i++)
   {
      printf("inserisci il %do valore: ", i+1);
      scanf("%d", &a[i]);
   }

   for (i=0, j=n-1; i<j; i++, j--)
   {
      t = a[i];
      a[i] = a[j];
      a[j] = t;
   }

   for (i=0; i<n; i++)
      printf("%d ", a[i]);
   printf("\n");

   return EXIT_SUCCESS;
}


/* ================================================================ */
/*   4                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#define MAXNUM 100

int main()
{
   int a[MAXNUM], t, *p, *q;
   int n;

   do
   {
      printf("Quanti valori? ");
      scanf("%d", &n);
   }while (n<0 || n>MAXNUM);

   q = a+n;  /* punta all'elemento successivo all'ultimo */
   for (p=a; p<q; p++)
   {
```

```
      printf("inserisci il %do valore: ", p-a+1);
      scanf("%d", p);
   }

   for (p=a, q=a+n-1; p<q; p++, q--) /* a+n-1 punta all'ultimo */
   {
      t = *p;
      *p = *q;
      *q = t;
   }

   q = a+n;
   for (p=a; p<q; p++)
      printf("%d ", *p);
   printf("\n");

   return EXIT_SUCCESS;
}


/* ================================================================ */
/*   5                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXSTR 100

int main()
{
   char s[MAXSTR], *p;
   printf("Introduci la stringa ");
   gets(s);

   if ((p=strchr(s, 'A')) != NULL && p - s < 10)
      printf("Compare almeno una A tra i primi 10 caratteri\n");
   else
      printf("Non compare nessuna A tra i primi 10 caratteri\n");

   return EXIT_SUCCESS;
}

/* Altra soluzione */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXSTR 100

int main()
{
   char s[MAXSTR]="", *p;
   printf("Introduci la stringa ");
   gets(s);

   if ((p=memchr(s, 'A', 10)) != NULL)
      printf("C'e' almeno una A tra i primi 10 caratteri\n");
   else
      printf("Non ci sono A tra i primi 10 caratteri\n");

   return EXIT_SUCCESS;
}
```

```
/* Nota
   Il parametro 10 nella memchr limita la ricerca ai primi 10
   caratteri, ma poiche' la stringa introdotta potrebbe essere piu'
   corta e la memchr comunque scandisce 10 caratteri, bisogna essere
   sicuri che oltre il \0 non ci sia una A dovuta ad una allocazione
   precedente della memoria, per questo si inizializza s a tutti \0
*/


/* ================================================================ */
/*   6                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXNUM 100

int main()
{
   char a[MAXNUM], b[MAXNUM], *p, *q;
   int i, spazio, n, nn;

   printf("Dammi la prima stringa: ");
   gets(a);
   printf("Dammi la seconda stringa: ");
   gets(b);

   if (strlen(a)+strlen(b)+1 > MAXNUM)
   {
      printf("La lunghezza finale eccede il massimo: %d\n", MAXNUM);
      return EXIT_FAILURE;
   }

   p=a+strlen(a);
   q=b;
   while ( (*p++ = *q++) != '\0')
      ;

   printf("%s\n", a);

   return EXIT_SUCCESS;
}


/* ================================================================ */
/*   7                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXNUM 100

int main()
{
   char a[MAXNUM], b[MAXNUM], *p, *q;

   printf("Dammi la prima stringa piu' lunga: ");
   gets(a);
   printf("Dammi la seconda stringa piu' corta: ");
   gets(b);
```

```
   p = a + strlen(a)-1; /* p punta all'ultimo carattere di a */
   q = b + strlen(b)-1; /* q punta all'ultimo carattere di b */

   while ( (q>=b && *p == *q) )
   {
      p--;
      q--;
   }
   if (q>b)
      printf("Non e' la parte terminale\n");
   else
      printf("E' la parte terminale\n");

   return EXIT_SUCCESS;
}


/* ================================================================ */
/*   8                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#define MAXNUM 100

int main()
{
   int a[MAXNUM], t, *p, *q;
   int n;

   do
   {
      printf("Quanti valori? ");
      scanf("%d", &n);
   }while (n<0 || n>MAXNUM);

   q = a+n;  /* punta all'elemento successivo all'ultimo */
   for (p=a; p<q; p++)
   {
      printf("inserisci il %do valore: ", p-a+1);
      scanf("%d", p);
   }

   for (p=a; p<a+n-1; p++)
      for (q=p+1; q<a+n; q++)
         if (*p > *q)
            t=*p, *p=*q, *q=t;
   q = a+n;
   for (p=a; p<q; p++)
      printf("%d ", *p);
   printf("\n");

   return EXIT_SUCCESS;
}


/* ================================================================ */
/*   9                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>
#define MAXNUM 100

int main()
{
   char v[MAXNUM], *p, *q;

   printf("Introduci la stringa: ");
   gets(v);

   p = v;
   q = v+(int)( (strlen(v)+1)/2.0 ) ; /* 1o char della 2a meta' */

   while (*q != '\0' && *p++ == *q++)
     ;

   if (*--p == *--q)
      printf("E' composta da due parti uguali\n");
   else
      printf("Non e' composta da due parti uguali\n");

   return EXIT_SUCCESS;
}


/* ================================================================ */
/*  10                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXSTR 100

int main()
{
   char a[MAXSTR], b[MAXSTR], t[MAXSTR], *p;
   int n;

   printf("Introduci la prima stringa: ");
   gets(a);
   printf("Introduci la seconda stringa: ");
   gets(b);

   if (strlen(a) < strlen(b)) /* a contiene sempre la piu' lunga */
   {
      strcpy(t, a);
      strcpy(a, b);
      strcpy(b, t);
   }

   n = 0;
   p = a;
   while ((p=strstr(p, b)) != NULL)
   {
      p++; /* sposta il punto di partenza oltre il primo
              carattere ogni volta */
      n++; /* numero di volte che viene trovato */
   }

   printf("La stringa piu' corta compare %d volte in quella "
          "piu' lunga\n", n);
```

```
   return EXIT_SUCCESS;
}


/* ================================================================ */
/*  11                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAXNUM 100

int main()
{
   char file[FILENAME_MAX], a[MAXNUM], *p, *q;
   int nn, l;
   char c;
   FILE *fp, *fp2;

   printf("Nome del file da leggere: ");
   gets(file);
   if ( (fp=fopen(file, "r")) == NULL)
   {
      printf("Non posso aprire il file %s\n", file);
      return EXIT_FAILURE;
   }

   printf("Nome del file da scrivere: ");
   gets(file);
   if ( (fp2=fopen(file, "w")) == NULL)
   {
      printf("Non posso aprire il file %s\n", file);
      return EXIT_FAILURE;
   }

   printf("Massima lunghezza: ");
   scanf("%d", &nn);

   while (fgets(a, MAXNUM,fp) != NULL)  /* mette in a anche il \n */
   {
      p = a;
      while (*p != '\0')
      {
         l = strlen(p);
         if (*(p+l-1)=='\n')
         {
            *(p+l-1) = '\0';        /* toglie il \n, se c'e' */
            l--;
         }

         if (l<=nn)
         {
            fputs(p, fp2);
            fputs("\n", fp2);
            break;             /* finita la riga */
         }

         /* entra qui se la stringa e' > di nn */
         q = p+nn; /* punta al chr successivo all'ultimo degli nn */
         while (*q != ' ' && q>=p)   /* cerca indietro lo spazio */
            q--;
```

```
        if (*q == ' ') /* spazio trovato? */
        {
           *q = '\0';        /* sostituisce \0 allo spazio cosi'
                                termina la stringa */
           fputs(p, fp2);
           fputs("\n", fp2);
           p = q+1;         /* p punta ora al primo carattere dopo
                               lo spazio tolto */
           continue;
        }
        else                /* spazio non trovato, spezza alla
                               posizione nn */
        {
           q = p+nn;        /* punta al char successivo all'ultimo
                               degli nn */
           c = *q;          /* salva in c il chr alla posiz. q */
           *q = '\0';       /* sostituisce \0 allo spazio cosi'
                               termina la stringa */
           fputs(p, fp2);
           fputs("\n", fp2);
           *q = c;          /* ripristinina il chr alla posiz. q */
           p = q;
        }
      }  /* while che spezza una riga */
   } /* while di input delle righe*/
   fclose(fp);
   fclose(fp2);

   return EXIT_SUCCESS;
}
```

1. Si scriva un programma che per 10 volte chieda all'utente un valore e ne calcoli il logaritmo in base 2.
   Per il calcolo del logaritmo si scriva una funzione con prototipo:
   ```
   double log2(double a);
   ```
   che calcoli il valore utilizzando la formula:

$$\log_2 x = \frac{\log_e x}{\log_e 2}$$

2. Si scriva un programma che chieda all'utente 10 valori e di questi calcoli la radice quadrata.
   Per il calcolo della radice quadrata si scriva una funzione con prototipo:
   ```
   double radice(double a, double prec);
   ```
   che calcoli il valore approssimato della radice quadrata di a con il metodo di Newton:

$$x_{i+1} = \frac{1}{2}\left(x_i + \frac{a}{x_i}\right)$$

   x_i sono approssimazioni successive della radice quadrata di a. Si assuma $x_0 = a$ e si iteri fintanto che $x_i - x_{i+1} >$ `prec`.

3. Si scriva una funzione con prototipo:
   `double media(double v[], int len);`
   che calcoli e restituisca la media di un vettore di `double` passato come argomento.
   Si scriva un programma che riempia due vettori di lunghezza differente (es. `a[8]` e `b[10]`, li passi a `media` e visualizzi il risultato per ciascuno di essi. La funzione non esegua operazioni di input/output.
4. Si scriva una funzione con prototipo:
   `void rovescia(char s[]);`
   che rovesci la stringa passata come argomento (modifica del parametro). Si scriva un programma che chieda una stringa, la passi a `rovescia` e la visualizzi.
5. Si scriva una funzione con prototipo:
   `int contastr(char a[], char x[]);`
   che conti quante volte la stringa `x` sia contenuta in `a`. N.B. "bb" in "bbb": 2 volte
6. Si scriva una funzione `undup` che modifichi una stringa eliminandone i caratteri duplicati: esempio: "ciao come_va?" → "ciao mev?"

7. Si scriva una funzione con prototipo:
```
void ordina(int v[], int len,
            int ord);
```
che ordini in senso crescente e decrescente il vettore di `int` passato come argomento.
Il senso dell'ordinamento venga indicato dal parametro `ord` (decrescente=0, crescente=1).
Si scriva un main di test.

8. Si scriva la funzione `sommaVett` che calcoli la somma di due vettori. Si scriva un `main` che chieda la dimensione dei vettori (max 100), ne chieda i valori, li passi alla funzione e visualizzi il vettore dei risultati. Non si alteri il contenuto dei due vettori da sommare. La funzione non faccia input/output.
Per non usare variabili esterne o variabili locali `static`, alla funzione bisogna passare anche il vettore dei risultati (il contenuto iniziale non è rilevante).

9. Un file di testo denominato `Parole.txt` contiene una lista di parole, una per riga. Non è noto a priori di quante righe sia composto. Si scriva un programma che chieda all'utente di introdurre una parola e visualizzi tutte le parole presenti nel file che anagrammate danno la parola introdotta. Si scrivano due funzioni: una che riordina alfabeticamente le lettere di una stringa ("telefono" → "`eeflnoot`") e un'altra che modifichi una stringa trasformandone tutti i caratteri in minuscolo.

10. Si scriva una funzione con prototipo:

```
double media(double M[][10],
             int righe, int colonne);
```

che calcoli e restituisca la media di una generica matrice di 10 colonne passata come argomento. Si scriva un programma che riempia due matrici di dimensioni richieste all'utente (massime 8x10 e 12x10, il massimo di 10 colonne ciascuna è per coerenza con il prototipo), le passi a `media` e visualizzi il risultato per ciascuna di esse.
La funzione non faccia input/output.

11. Scrivere una funzione con prototipo

```
int parseToIntVect(char *stringa,
                   int vett[], int n);
```

che analizzi la `stringa` data estraendo da essa i primi `n` valori numerici interi e li inserisca nel vettore `vett[]`. Se vi sono più di `n` valori, i successivi vengano ignorati. La funzione restituisca il numero di elementi letti (che possono quindi essere meno di `n`). Si supponga che il file non contenga caratteri diversi da cifre e whitespace. Si scriva un main di test.

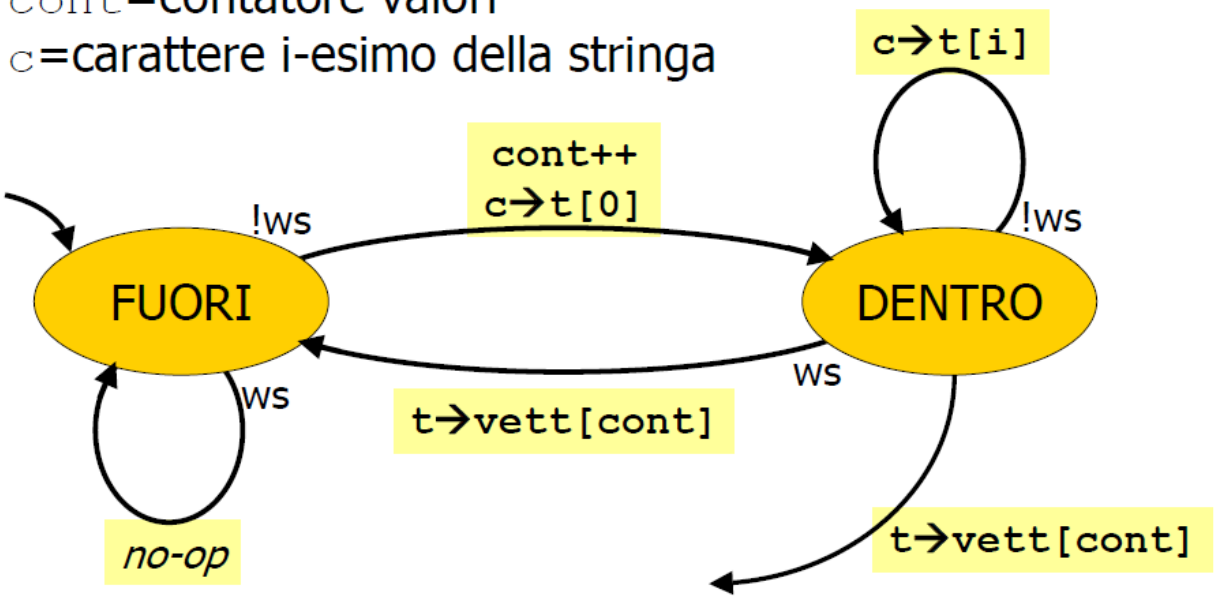
Soluzione a stati

ws=white space

`t`=stringa per contenere i caratteri da trasf. in numero

`cont`=contatore valori

`c`=carattere i-esimo della stringa



12. Si scriva un programma che permetta di calcolare il prodotto di 2 numeri interi senza segno introdotti dall'utente e composti ciascuno da un massimo di 1000 cifre. Conviene modularizzare il codice utilizzando opportune funzioni di supporto (ad esempio per moltiplicare un numero per una cifra, per fare lo shift di n posizioni a sinistra di un numero e per sommare due numeri).

Soluzioni:

```
/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#define NUM 10

double log2(double valore);

int main()
{
   int i;
   double a;

   for (i=0; i<NUM; i++)
   {
      printf("Introdurre un numero: ");
      scanf("%lf", &a);
      printf("Il log2 di %f vale: %f\n", a, log2(a));
   }

   return EXIT_SUCCESS;
}

/*-------------------------------------------------------------*/
double log2(double x)
{
   return log(x)/log(2.0);
}


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */
```

```c
#include <stdio.h>
#include <stdlib.h>
#define NUM 10

double radice(double valore, double precisione);

int main()
{
   int i;
   double x, e;

   for (i=0; i<NUM; i++)
   {
      printf("Introdurre un numero: ");
      scanf("%lf", &x);
      printf("Introdurre la precisione: ");
      scanf("%lf", &e);
      printf("Radice quadrata di %f = %.15f\n", x, radice(x,e));
   }

   return EXIT_SUCCESS;
}


/*-------------------------------------------------------*/
double radice(double a, double prec)
{
   double xn; /* x nuova approssimazione */
   double xp; /* x precedente approssimazione */
   xn = a;
   do
   {
      xp = xn;
      xn = 0.5 * (xp + a/xp);
   }while (xp-xn > prec);

   return xn;
}


/* ============================================================ */
/*   3                                                          */
/* ============================================================ */

#include <stdio.h>
#include <stdlib.h>
#define DIM1 8
#define DIM2 10

double media(double vettore[], int lunghezza);

int main()
{
   int i;
   double a[DIM1], b[DIM2];

   printf("Inserire %d valori per a:\n", DIM1);
   for (i=0; i<DIM1; i++)
      scanf("%lf", &a[i]);
   printf("Inserire %d valori per b:\n", DIM2);
   for (i=0; i<DIM2; i++)
      scanf("%lf", &b[i]);
```

```
   printf("La media di a vale: %f\n", media(a,DIM1));
   printf("La media di b vale: %f\n", media(b,DIM2));

   return EXIT_SUCCESS;
}


/*-------------------------------------------------------*/
double media(double v[], int len)
{
   double somma=0;
   int i;

   for (i=0; i<len; i++)
      somma += v[i];

   return somma/len;
}


/* ================================================================ */
/*   4                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXNUM 100

void rovescia(char []);   /* solo il nome puo' essere tralasciato */

int main()
{
   char v[MAXNUM];

   printf("Introduci la stringa: ");
   gets(v);

   rovescia(v);
   printf("%s\n", v);

   return EXIT_SUCCESS;
}


/*-------------------------------------------------------*/
void rovescia(char s[])
{
   int i, l;
   char t;

   l=strlen(s);
   for (i=0; i<l/2; i++)
   {
      t=s[i];
      s[i]=s[l-1-i];
      s[l-1-i]=t;
   }
}

/* Note
   l-1-i e' l'indice dell'elemento i-esimo contato da destra.
```

```
   Il ciclo FOR poteva essere anche questo (bisogna definire j)
   for (i=0, j=l-1; i<j; i++, j--)
   {
      t=s[i];
      s[i]=s[j];
      s[j]=t;
   }
*/

/* Altra soluzione */

void rovescia(char s[])
{
   int i, j, l;
   char temp[MAXNUM];

   l=strlen(s);
   for (i=0, j=l-1; i<l; i++, j--)
      temp[i]=s[j];
   temp[l] = '\0';
   strcpy(s, temp);
}

/* Nota
   Crea una nuova stringa prelevando i caratteri dalla fine della
   stringa data, la ricopia in s perche' solo s e' condivisa con il
   chiamante
*/


/* ================================================================ */
/*   5                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXNUM 100

int contastr(char *stringa_dove_cercare, char *stringa_da_cercare);

int main()
{
   char v[MAXNUM], w[MAXNUM];

   printf("Introduci la stringa dove cercare: ");
   gets(v);
   printf("Introduci la stringa da cercare: ");
   gets(w);

   printf("La stringa e' stata trovata %d volte\n", contastr(v,w));

   return EXIT_SUCCESS;
}

/*-------------------------------------------------------*/
int contastr(char *a, char *x)
{
   int lx, cont=0;

   lx=strlen(x);
   while (*(a+lx-1) != '\0')   /* ultimo chr di a valutato da */
      if (strncmp(a++, x, lx) == 0)                /* strncmp */
```

```
         cont++;

   return cont;
}

/* soluzione alternativa
int contastr(char *a, char *x)
{
   int i, j, la, lx, cont=0;

   la=strlen(a);
   lx=strlen(x);
   for (i=0; i<la-lx+1; i++)
   {
      for (j=0; j<lx; j++)
         if(a[i+j]!=x[j])
            break;  /* c'e' un chr non corrispondente */
      if (j==lx)    /* vero se non ha mai eseguito break */
         cont++;
   }
   return cont;
}
*/


/* ================================================================ */
/*   6                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXNUM 100

void undup(char *);

int main()
{
   char stringa[MAXNUM];

   printf("Introduci la stringa: ");
   gets(stringa);

   undup(stringa);
   printf("La stringa senza i duplicati e':\n%s\n", stringa);

   return EXIT_SUCCESS;
}


/*-------------------------------------------------------*/
void undup(char *s)
{
   int i, j;
   int l;
   int k; /* indice dell'ultima posizione con chr non duplicati */


   l = (int)strlen(s);
   k = 0; /* il primo non puo'essere duplicato e ha indice 0 */
   for (i=1; i<l; i++)
   {
      /* cerca nei precedenti a k (incluso) */
```

```
      for (j=k; j>=0 && s[j] != s[i]; j--)
         ;
      if (j<0)            /* se non lo trova, j arriva a -1 */
         s[++k] = s[i];
   }
   s[++k] = '\0';   /* termina la stringa */
}


/* Altra soluzione */
void undup(char *s)
{
   int i;
   int l; /* lunghezza stringa in input */
   int k; /* indice dell'ultima posizione con chr non duplicati */
   char car[256] = {0}; /* tutti i flag sono inizializzati a 0 */

   l = (int)strlen(s);
   k = -1; /* si posiziona prima del primo carattere */
   for (i=0; i<l; i++)
   {
      if (car[(int)s[i]] == 0)  /* considera il flag relativo al */
      {                         /*   carattere s[i] */
         car[(int)s[i]] = 1;    /* imposta il flag a 1 */
         s[++k] = s[i];         /* lo copia alla posizione ++k */
      }
   }
   s[++k] = '\0';   /* termina la stringa */
}
/* Nota
   Ogni elemento del vettore car corrisponde ad un carattere del
   codice ASCII; car[65] e' l'elemento corrispodnente alla 'A'.
   car[] e' un vettore di flag e ogni elemento vale 1 se e' già stata
   incontrata la lettera con quel codice ASCII (car[65] vale 1 se si e'
   gia' trovata una 'A'), mentre 0 vuol dire che quel carattere non e'
   stato ancora trovato. Questo algoritmo e' piu' veloce in quanto
   non c'e' la ricerca: valuta solo il flag corrispondente
*/

/* Altra soluzione */
void undup(char *s)
{
   int i, j, k;
   int l;

   l = (int)strlen(s);
   for (i=1; i<l; i++)
   {
      /* cerca nei precedenti a i (escluso) */
      for (j=i-1; j>=0 && s[j] != s[i]; j--)
         ;
      if (j>=0)            /* se lo trova, j e' >= 0 */
         s[i] = '\0';
   }
   /* ora copia tutti i caratteri escludendo quelli di codice \0 */
   k=-1;
   for (i=0; i<l; i++)
      if (s[i] != '\0')
         s[++k]=s[i];
   s[++k] = '\0';   /* termina la stringa */
}
/* Nota
   In questa soluzione, ogni carattere ripetuto viene sostituito
```

```
   con un carattere che non si suppone l'utente introduca, ad
   esempio il carattere \0. Questo comporta una limitazione all'uso
   del programma (ossia non funziona con stringhe _binarie_ che
   contengono un carattere \0, una stringa normale non contiene il
   carattere \0 che alla fine)
*/



/* ================================================================ */
/*   7                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#define MAXNUM 10

void ordina(int vettore[], int lunghezza, int ordinamento);

int main()
{
   int v[MAXNUM];
   int i, l, ord;

   printf("Quanti elementi? ");
   scanf("%d", &l);

   for (i=0; i<l; i++)
   {
      printf("Elemento %d: ", i+1);
      scanf("%d", &v[i]);
   }

   do
   {
      printf("Ordinamento crescente [1] o decrescente [0]? ");
      scanf("%d", &ord);
   }while(ord != 0 && ord != 1);

   ordina(v,l,ord);
   for (i=0; i<l; i++)
      printf("Elemento %d: %d\n", i+1, v[i]);

   return EXIT_SUCCESS;
}


/*-----------------------------------------------------------*/
void ordina(int v[], int len, int ord)
{
   int i, j;
   int t;

   for (i=0; i<len-1; i++)
      for (j=i+1; j<len; j++)
         if (ord==1 && v[i] > v[j])
         {
            t=v[i];
            v[i]=v[j];
            v[j]=t;
         }
         else if (ord==0 && v[i] < v[j])
         {
```

```
            t=v[i];
            v[i]=v[j];
            v[j]=t;
         }
}

/* Nota
   La funzione ordina si potrebbe condensare in questa:

void ordina(int v[], int len, int ord)
{
   int i, j;
   int t;

   for (i=0; i<len-1; i++)
      for (j=i+1; j<len; j++)
         if (ord==1 && v[i] > v[j] || ord==0 && v[i] < v[j])
         {
            t=v[i];
            v[i]=v[j];
            v[j]=t;
         }
}

   Se poniamo:
     X = (ord==1)
     Y = (v[i] < v[j])
   la condizione puo' essere scritta
     X && !Y || !X && Y
   (in algebra booleana si scrive XY'+X'Y).
   E' vero che !(v[i]<v[j]) equivale a v[i]>=v[j] e non a v[i]>v[j],
   ma l'effetto di questa differenza e' lo scambio dei valori
   uguali, cosa che aggiunge una trascurabile inefficienza.
   L'espressione booleana XY'+X'Y equivale a: A EXOR B.
   In C non esiste l'operatore logico EXOR, tuttavia esiste quello
   bitwise che può essere utilizzato se X e Y valgono 0 o 1. Essendo
   questo il caso (infatti X e Y sono il risultato di un confronto),
   la condizione finale puo' essere scritta:
     (ord==1) ^ (v[i]<v[j])
   Nel caso la velocita' sia un requisito fondamentale, la soluzione
   migliore e' quella con il minor numero di controlli; in questo
   caso dunque conviene duplicare tutto il codice separando i due
   ordinamenti crescente e decrescente.
*/


/* ================================================================ */
/*   8                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#define MAXNUM 100

void sommaVett(const int a[], const int b[], int r[], int n);

int main()
{
   int x[MAXNUM];
   int y[MAXNUM];
   int risultato[MAXNUM];
   int n;
   int i;
```

```
   do {
      printf("Lunghezza dei vettori? ");
      scanf("%d", &n);
   }while (n<1 || n>MAXNUM);

   for (i=0; i<n; i++)
   {
      printf("x(%d): ", i+1);
      scanf("%d", &x[i]);
   }

   for (i=0; i<n; i++)
   {
      printf("y(%d): ", i+1);
      scanf("%d", &y[i]);
   }

   sommaVett(x, y, risultato, n);

   for (i=0; i<n; i++)
      printf("risultato(%d): %d\n", i+1, risultato[i]);

   return EXIT_SUCCESS;
}


/*-------------------------------------------------------*/
void sommaVett(const int a[], const int b[], int r[], int n)
{
   int i;

   for (i=0; i<n; i++)
      r[i] = a[i] + b[i];
}

/* Note
   Il vettore da riempire con il risultato della somma viene
    allocato nel chiamante (main) e passato alla funzione, il
    contenuto iniziale di questo vettore e' irrilevante.
   Per garantire la non modifica dei vettori si puo' usare la
    clausola const come indicato.
*/

/* ================================================================ */
/*   9                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX 100
#define FILENAME "C:\\Parole.txt"  /* MODIFICARE OPPORTUNAMENTE */

void ordina(char []);
void inMinuscolo(char[]);

int main()
{
   char stringa[MAX], parola[MAX], parolaOrdinata[MAX];
   FILE *fp;
```

```
   printf("Parola di cui cercare gli anagrammi: ");
   gets(stringa);
   inMinuscolo(stringa); /* la mette in minuscolo */
   ordina(stringa);      /* riordina alfabeticamente i caratteri */

   if ( (fp=fopen(FILENAME,"r")) == NULL )
   {
      fprintf(stderr, "Non posso aprire il file %s\n", FILENAME);
      return EXIT_FAILURE;
   }

   if (fscanf(fp, "%s", parola) == EOF) /* primo valore */
   {
      fprintf(stderr, "File %s vuoto\n", FILENAME);
      return EXIT_FAILURE;
   }

   do
   {
      strcpy(parolaOrdinata, parola);
      inMinuscolo(parolaOrdinata); /* la mette in minuscolo */
      ordina(parolaOrdinata);      /* riordina alfabet. i chr */
      if (strcmp(stringa,parolaOrdinata) == 0)
         printf("%s\n", parola);
   }while(fscanf(fp, "%s", parola) != EOF);

   return EXIT_SUCCESS;
}


/*-------------------------------------------------------*/
void inMinuscolo(char s[])
{
   int i;
   for (i=0; s[i] != '\0'; i++)
      s[i] = (char)tolower(s[i]);
}


/*-------------------------------------------------------*/
void ordina(char s[])
{
   int i, j, jmin, len;
   char t;

   len = strlen(s);
   for (i=0; i<len-1; i++)
   {
      jmin = i;
      for (j=i+1; j<len; j++)
         if (s[j] < s[jmin])
            jmin = j;

         t=s[jmin];
         s[jmin]=s[i];
         s[i]=t;
   }
}


/* ================================================================ */
/*  10                                                              */
/* ================================================================ */
```

```c
#include <stdio.h>
#include <stdlib.h>
#define MAXCOLONNE 10
#define MAXRIGHE1 8
#define MAXRIGHE2 12

double media(double M[][MAXCOLONNE], int righe, int colonne);

int main()
{
   int i, j;
   double Mx1[MAXRIGHE1][MAXCOLONNE], Mx2[MAXRIGHE2][MAXCOLONNE];
   int righe1, colonne1, righe2, colonne2;

   do {
      printf("Numero righe matrice 1 (max=%d): ", MAXRIGHE1);
      scanf("%d", &righe1);
   }while (righe1<1 || righe1>MAXRIGHE1);
   do {
      printf("Numero colonne matrice 1 (max=%d): ", MAXCOLONNE);
      scanf("%d", &colonne1);
   }while (colonne1<1 || colonne1>MAXCOLONNE);
   do {
      printf("Numero righe matrice 2 (max=%d): ", MAXRIGHE2);
      scanf("%d", &righe2);
   }while (righe2<1 || righe2>MAXRIGHE2);
   do {
      printf("Numero colonne matrice 2 (max=%d): ", MAXCOLONNE);
      scanf("%d", &colonne2);
   }while (colonne2<1 || colonne2>MAXCOLONNE);

   printf("Inserire i valori per Mx1:\n");
   for (i=0; i<righe1; i++)
      for (j=0; j<colonne1; j++)
      {
         printf("Mx1[%d][%d]: ", i, j);
         scanf("%lf", &Mx1[i][j]);
      }

   printf("Inserire i valori per Mx2:\n");
   for (i=0; i<righe2; i++)
      for (j=0; j<colonne2; j++)
      {
         printf("Mx2[%d][%d]: ", i, j);
         scanf("%lf", &Mx2[i][j]);
      }
   printf("La media di Mx1 e': %f\n", media(Mx1,righe1, colonne1));
   printf("La media di Mx2 e': %f\n", media(Mx2,righe2, colonne2));

   return EXIT_SUCCESS;
}


/*-------------------------------------------------------*/
double media(double M[][MAXCOLONNE], int righe, int colonne)
{
   double somma=0;
   int i, j;

   for (i=0; i<righe; i++)
      for (j=0; j<colonne; j++)
       somma += M[i][j];
```

```
   return somma/(righe*colonne);
}


/* ================================================================ */
/*  11                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#define MAXVETT 10
#define MAXSTRING 80

int parseToIntVect(char *stringa, int vett[], int n);

int main()
{
   int v[MAXVETT];
   char s[MAXSTRING];
   int i, n;

   printf("Introdurre stringa da analizzare:\n");
   gets(s);
   n = parseToIntVect(s, v, MAXVETT);
   printf("Letti %d valori:\n", n);
   for (i=0; i<n; i++)
      printf("%d\n", v[i]);
   return EXIT_SUCCESS;
}

/*-----------------------------------------------------------*/
int parseToIntVect(char *stringa, int vett[], int n)
{
   int i, k, l, cont;
   char t[MAXSTRING];

   cont=0;
   l=strlen(stringa);
   i=0;
   do
   {
      /* salta whitespace iniziali del campo */
      while (i<l && isspace(stringa[i]))
         i++;

      /* accumula i caratteri fino al prossimo whitespace */
      k=0;
      while (i<l && !isspace(stringa[i]))  /* anche il - e' letto */
         t[k++]=stringa[i++];

      if (k>0) /* se k==0 significa che non ha letto alcun valore */
         t[k]='\0';
      vett[cont++]=atoi(t);
   }while(i<l && cont<n);

   return cont;
}

/* Altra soluzione (a stati) */

int parseToIntVect(char *stringa, int vett[], int n)
```

```
{
   enum {FUORI, DENTRO} stato;
   int i, k, l, cont;
   char t[MAXSTRING];

   stato = FUORI;

   cont=0;
   l=strlen(stringa);
   for (i=0; i<l && cont < n; i++)
      switch(stato)
      {
      case FUORI:
         if (!isspace(stringa[i])) /* entra DENTRO */
         {
            k=0;
            t[k++]=stringa[i];
            stato = DENTRO;
         }
         break;
      case DENTRO:
         if (!isspace(stringa[i])) /* resta dentro */
            t[k++]=stringa[i];
         else                      /* esce FUORI */
         {
            t[k]='\0';
            vett[cont++]=atoi(t);
            stato = FUORI;
         }
         break;
      }
      if (stato == DENTRO) /* se la stringa finisce con un numero */
      {
         t[k]='\0';
         vett[cont++]=atoi(t);
      }

   return cont;
}

/* Nota
   Si usa un approccio a stati: DENTRO un numero e FUORI da un
   numero, si leggono i caratteri uno per uno e a seconda dello
   stato viene intrappresa una diversa azione:
   - se e' FUORI:
     - se trova un non-whitespace entra DENTRO, copia il carattere
       in t[0]
     - altrimenti non fa nulla
   - se e' DENTRO:
     - se trova un non-whitespace copia il carattere in t (e
       resta DENTRO)
     - altrimenti termina t, la passa ad atoi(), mette il valore
       in vett[cont], incrementa cont, esce FUORI
   Se la stringa termina con un numero, bisogna terminare la
   stringa, mettere l'ultimo valore in vett[] e incrementare cont
*/


/* ================================================================ */
/*  12                                                              */
/* ================================================================ */

#include <stdio.h>
```

```c
#include <stdlib.h>
#include <string.h>
#define MAXCIFRE 100

void moltiplica(char a[], int la, char x, char r[]);
void aggiungiZeri(char a[], int la, int quanti);
void somma(char a[], char b[], int cifre);

int main()
{
   int i, j;
   char p[MAXCIFRE+1], s[MAXCIFRE+1], q;
   char ris[MAXCIFRE*2+1]={0}, t[MAXCIFRE*2+1]={0};
   int lp, ls, lris;

   printf("Primo numero (max %d cifre): ", MAXCIFRE);
   gets(p);
   printf("Secondo numero (max %d cifre): ", MAXCIFRE);
   gets(s);
   lp=strlen(p);
   ls=strlen(s);
   lris = lp+ls; /* la lunghezza di <ris> e' al massimo lp+ls */

   /* poiche' il risultato ha in genere piu' cifre e poiche' una
      stringa puo' essere ampliata solo al fondo, si inverte
      la posizione di ciascuna cifra cosi' che p[0] contenga la
      cifra meno significativa (LSD), ossia quella di potenza 0,
      lo stesso viene fatto per la stringa s */
   for (i=0, j=lp-1; i<lp/2; i++, j--)
      q=p[i], p[i]=p[j], p[j]=q;
   for (i=0, j=ls-1; i<ls/2; i++, j--)
      q=s[i], s[i]=s[j], s[j]=q;

   /* trasforma ogni carattere di p e di s nel valore decimale */
   for (i=0; i<lp; i++)
      p[i] -= '0';
   for (i=0; i<ls; i++)
      s[i] -= '0';

   for (i=0; i<ls; i++) /* per ciascuna cifra di s dal LSD al MSD */
   {
      moltiplica(p, lp, s[i], t);
      aggiungiZeri(t, lp+1, i);    /* dopo la moltiplicazione la max
                                      lunghezza di t è <lp+1> */
      somma(ris, t, lp+i+1); /* all'iterazione i, <lp+i+1> e' la max
                                lunghezza sia di ris sia di t */
   }

   /* il numero di cifre finale e' composto o da <lp+ls> o da
      <lp+ls-1> cifre, per conoscerlo basta verificare se la prima
      cifra (MSD, che pero' e' a destra perche' invertita) e' 0 */
   if (ris[lris-1]==0)
      lris--;
   /* solo se il risultato e' zero, il primo carattere e' 0, quindi
      si eliminano tutti gli altri zeri impostando lris a 1 per
      visualizzare uno zero solo */
   if (ris[lris-1]==0)
      lris=1;
   /* ri-inverte la posizione dei caratteri in modo che il primo sia
      il MSD, notare che la stringa e' gia' terminata da un \0
      grazie all'inizializzazione di ris */
   for (i=0, j=lris-1; i<lris/2; i++, j--)
      q=ris[i], ris[i]=ris[j], ris[j]=q;
```

```
   /* riconverte in stringa il numero ris, trasformando in carattere
      ogni cifra di ris */
   for (i=0; i<lris; i++)
      ris[i] += '0';

   printf("Risultato: %s\n", ris);

   return EXIT_SUCCESS;
}


/*-----------------------------------------------------------*/
/* moltiplica <a> per <x> e mette il risultato in <r>, serve passare
   la lunghezza di <a> in quanto avendo trasformato i caratteri '0'
   in valori 0, questi coincidono con il '\0' e quindi non si puo'
   piu' usare strlen */
void moltiplica(char a[], int la, char x, char r[])
{
   int i, m, riporto;

   /* non serve azzerare <r>, poiche' la moltiplicazione produce
      tante cifre quante sono quelle di <la> e l'eventuale altra
      cifra viene impostata dall'ultimo riporto (eventualmente 0) */
   riporto=0;
   for (i=0; i<la; i++)
   {
      m=a[i]*x+riporto;
      r[i]=(char)(m%10);
      riporto=m/10;
   }
   r[la]=(char)riporto;
}


/*-----------------------------------------------------------*/
/* aggiunge <quanti> zeri a destra del numero <a>,
   essendo stato capovolto (LSD in a[0]), questo si ottiene
   aggiungendo degli zeri all'inizio della stringa */
void aggiungiZeri(char a[], int la, int quanti)
{
   int i;

   for (i=la-1; i>=0; i--)
      a[i+quanti]=a[i];
   for (i=quanti-1 ; i>=0; i--)
      a[i]=(char)0;
}


/*-----------------------------------------------------------*/
/* somma <a> e <b> e mette il risultato in <a>, si limita a <la>
   cifre (non ce ne sono di piu' in quanto <a> e' la somma dei
   risultati parziali, quindi <a> e' piu' lungo di <b>) */
void somma(char a[], char b[], int la)
{
   int i, riporto, m;

   riporto=0;
   for (i=0; i<la; i++)
   {
      m=a[i]+b[i]+riporto;
      a[i]=(char)(m%10);
```

```
        riporto=m/10;
    }
    a[la]=(char)riporto;
}
```

2. Si scriva un programma che dichiari il seguente tipo di dati che descrive un generico poligono regolare

```
struct poligono
{
    int nlati;
    double lato;
};
```

Si scrivano le seguenti funzioni e le si chiamino da un `main` di prova:

1. `struct poligono creapoli(void);`
   chiede all'utente il numero lati e la lunghezza lato, quindi restituisce una `struct poligono`
2. `double areapoli(struct poligono p);`
   calcola l'area del poligono passato

$$A = \frac{n \cdot l^2}{4 \cdot \tan \frac{\pi}{n}}$$

3. `double perimpoli(struct poligono p);`
   calcola il perimetro del poligono passato
4. `void doppiopoli(struct poligono *pp);`
   raddoppia il lato *dello stesso poligono passato*

3. Come il precedente, ma si usi la `typedef`

Soluzioni:

```
/* ================================================================ */
/*   2                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

/* definita esternamente perche' sia visibile a tutte le funzioni */
struct poligono
{
   int nlati;
   double lato;
```

```
};

struct poligono creapoli(void);
double areapoli(struct poligono poli);
double perimpoli(struct poligono poli);
void doppiopoli(struct poligono *ppoli);

int main()
{
   struct poligono poli1, poli2;

   poli1 = creapoli();

   printf("Il poligono 1 ha perimetro: %f e area %f\n",
                                perimpoli(poli1), areapoli(poli1));

   poli2=poli1;
   doppiopoli(&poli2);
   printf("Il poligono 2 ha perimetro: %f e area %f\n",
                                perimpoli(poli2), areapoli(poli2));

   return EXIT_SUCCESS;
}

struct poligono creapoli(void)
{
   struct poligono po;

   printf("Numero lati poligono: ");
   scanf("%d", &po.nlati);
   printf("Lunghezza lati: ");
   scanf("%lf", &po.lato);
   return po;
}

double areapoli(struct poligono poli)
{
   const double PI = 4.0 * atan(1.0);      /* pi greco */
   return (poli.nlati*poli.lato*poli.lato)/(4.0*tan(PI/poli.nlati));
}

double perimpoli(struct poligono poli)
{
   return poli.lato * poli.nlati;
}

void doppiopoli(struct poligono *ppoli)
{
   ppoli->lato *= 2;   /* equivalentemente: (*ppoli).lato *=2; */
}


/* ========================================================================== */
/*   3                                                                        */
/* ========================================================================== */

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

/* definita esternamente perche' sia visibile a tutte le funzioni */
typedef struct poligono     /* il tag poligono puo' essere omesso */
{
```

```
   int nlati;
   double lato;
} Poligono;

Poligono creapoli(void);
double areapoli(Poligono poli);
double perimpoli(Poligono poli);
void doppiopoli(Poligono *ppoli);

int main()
{

   Poligono poli1, poli2;
   poli1 = creapoli();

   printf("Il poligono 1 ha perimetro: %f e area %f\n", perimpoli(poli1),
areapoli(poli1));

   poli2=poli1;
   doppiopoli(&poli2);
   printf("Il poligono 2 ha perimetro: %f e area %f\n", perimpoli(poli2),
areapoli(poli2));

   return EXIT_SUCCESS;
}

Poligono creapoli(void)
{
   Poligono po;

   printf("Numero lati poligono: ");
   scanf("%d", &po.nlati);
   printf("Lunghezza lati: ");
   scanf("%lf", &po.lato);
   return po;
}

double areapoli(Poligono poli)
{
   const double PI = 4.0 * atan(1.0);      /* pi greco */
   return (poli.nlati*poli.lato*poli.lato)/(4.0*tan(PI/poli.nlati));
}

double perimpoli(Poligono poli)
{
   return poli.lato * poli.nlati;
}

void doppiopoli(Poligono *ppoli)
{
   ppoli->lato *= 2;   /* equivalentemente: (*ppoli).lato *=2; */
}
```

1. Scrivere una funzione ricorsiva per determinare se una stringa è palindroma.
2. Scrivere una funzione ricorsiva per stampare una stringa a rovescio (non la deve invertire, ma solo stampare).
3. Scrivere una funzione ricorsiva che determini il minimo di un vettore di interi.
5. Scrivere una funzione ricorsiva per cercare un valore all'interno di un vettore non ordinato (ricerca lineare). Risultato: –1 se non lo trova, altrimenti l'indice della posizione dove è stato trovato.
6. Scrivere una funzione ricorsiva per cercare un valore all'interno di un vettore ordinato (ricerca dicotomica). Risultato: – 1 o l'indice.
7. Scrivere una funzione che realizzi un *selection sort* (cerca il valore più piccolo e lo mette in testa, ecc.) in modo ricorsivo su un vettore di interi.

Soluzioni:

```
/* ================================================================ */
/*   1                                                              */
/* ================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXSTR 100

int palin(char *s, int lung);

int main()
{
```

```
   char stringa[MAXSTR];

   printf("Inserire la stringa: ");
   gets(stringa);

   if (palin(stringa, strlen(stringa)))
      printf("La stringa e' palindroma\n");
   else
      printf("La stringa non e' palindroma\n");

   return EXIT_SUCCESS;
}

int palin(char *s, int lung)
{
   if (lung <=1 )  /* caso di base */
      return 1;    /* e' palindroma */
   return *s == *(s+lung-1) && palin(++s, lung-2) ;
   /* da' 1 se palindroma, 0 altrimenti se i chr estremi sono uguali
      e la stringa centrale rimanente e' palindroma (riduz. della
      complessita') restituisce 1 */
}


/* Altra soluzione */

int palin(char *s, int lung)
{
   if (lung <=1 )              /* caso di base 1: e' palindroma */
      return 1;
   else if (*s != *(s+lung-1)) /* caso di base 2: non palindroma */
      return 0;
   return palin(++s, lung-2) ; /* se i chr estremi sono uguali e la
                                  stringa centrale rimanente e'
                                  palindroma (riduz. complessita' */
}


/* ================================================================ */
/*   2                                                              */
/* ================================================================ */


#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAXSTR 100

void stampa(char *s);

int main()
{
   char stringa[MAXSTR];

   printf("Inserire la stringa: ");
   gets(stringa);

   stampa(stringa);

   return EXIT_SUCCESS;
}

void stampa(char *s)
```

```
{
   if (*s == '\0')
      return;      /* caso di base */
   else
   {
      stampa(s+1); /* stampa il primo carattere di s+1 */
      putchar(*s); /* e poi quello di s */
      return;
   }
}


/* Altra soluzione (piu' compatta, ma identica) */

void stampa(char *s)
{
   if (*s != '\0')
   {
      stampa(s+1); /* stampa il primo carattere di s+1 */
      putchar(*s); /* e poi quello di s */
   }
}


/* ================================================================ */
/*   3                                                              */
/* ================================================================ */

/* NON TAIL */

#include <stdio.h>
#include <stdlib.h>
#define LEN 10

#define min(A,B) ((A)<(B))?(A):(B)

int cercaMin(int v[], int len);

int main()
{
   int vett[LEN], i;

   for (i=0; i<LEN; i++)
   {
      printf("Inserire il %do valore: ", i+1);
      scanf("%d", &vett[i]);
   }

   printf("Il minimo vale: %d\n", cercaMin(vett,LEN));

   return EXIT_SUCCESS;
}

int cercaMin(int v[], int len)
{
   int m;  /* serve la variabile m perche' chiama una macro */

   if (len==1)
      return v[0];
   else
   {
      m = cercaMin(v+1, len-1);
      return min( v[0], m );
```

```
   }
}


/* ================================================================ */
/*   5                                                              */
/* ================================================================ */

/* TAIL */

#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int cercaLin(int v[], int x, int len);

int main()
{
   int vett[MAX], x, i;

   for (i=0; i<MAX; i++)
   {
      printf("Inserire il %do valore: ", i+1);
      scanf("%d", &vett[i]);
   }

   printf("Valore da cercare: ");
   scanf("%d", &x);

   if (cercaLin(vett, x, MAX) >= 0)
      printf("Trovato\n");
   else
      printf("Non trovato\n");

   return EXIT_SUCCESS;
}

/* cerca dalla fine del vettore */
int cercaLin(int v[], int x, int len)
{
   if (len == 0)           /* caso di base: non trovato */
      return -1;
   else if (v[len-1] == x) /* caso di base: l'ultimo carattere e'
                              quello cercato */
      return len-1;
   else                    /* altrimenti accorcia la stringa,
                              riducendo la complessita' */
      return cercaLin(v, x, len-1);
}

/* versione con accumulatore static */
/* cerca dall'inizio del vettore */
/* notare che non c'e' l'accumulatore start tra i parametri */
int cercaLin(int v[], int x, int len)
{
   static int start = 0;   /* accumulatore static */
   if (start == len)       /* caso di base: non trovato */
      return -1;
   else if (v[start] == x) /* caso di base: il carattere e' quello
                              cercato in pos */
      return start;
   else                    /* altrimenti parte da start+1, riducendo
                              la complessita' */
```

```
   {
      start++;
      return cercaLin(v, x, len);   /* sempre uguali in tutte */
   }                                /*  le chiamate */
}

/* Altra soluzione */

#include <stdio.h>
#include <stdlib.h>
#define MAX 10

int cercaLin(int v[], int x, int len, int start);

int main()
{
   int vett[MAX], x, i;

   for (i=0; i<MAX; i++)
   {
      printf("Inserire il %do valore: ", i+1);
      scanf("%d", &vett[i]);
   }

   printf("Valore da cercare: ");
   scanf("%d", &x);

   if (cercaLin(vett, x, MAX, 1) >= 0)
      printf("Trovato\n");
   else
      printf("Non trovato\n");

   return EXIT_SUCCESS;
}

/* cerca dall'inizio del vettore */
/* start e' l'accumulatore */
int cercaLin(int v[], int x, int len, int start)
{
   if (start == len)       /* caso di base: non trovato */
      return -1;
   else if (v[start] == x) /* caso di base: il carattere e' quello
                              cercato in start */
      return start;
   else                    /* altrimenti parte da start+1, riducendo
                              la complessita' */
      return cercaLin(v, x, len, start+1);
}


/* ================================================================ */
/*   6                                                              */
/* ================================================================ */

/* TAIL */

#include <stdio.h>
#include <stdlib.h>
#define MAX 10

void ordina(int v[], int l);
int cercaDicot(int v[], int x, int left, int right);
```

```
int main()
{
   int vett[MAX], y, i;

   for (i=0; i<MAX; i++)
   {
      printf("Inserire il %do valore: ", i+1);
      scanf("%d", &vett[i]);
   }

   ordina(vett, MAX);

   printf("Valore da cercare: ");
   scanf("%d", &y);

   if (cercaDicot(vett, y, 0, MAX-1) >= 0)
      printf("Trovato\n");
   else
      printf("Non trovato\n");

   return EXIT_SUCCESS;
}

void ordina(int v[], int len)
{
   int i, j, jmin;
   char t;

   for (i=0; i<len-1; i++)
   {
      jmin = i;
      for (j=i+1; j<len; j++)
         if (v[j] < v[jmin])
            jmin = j;

         t=v[jmin];
         v[jmin]=v[i];
         v[i]=t;
   }
}

int cercaDicot(int v[], int x, int left, int right)
{
   int c;

   if (left > right)
      return -1;
   else
   {
      c=(left+right)/2;
      if (x > v[c])
         /* cerca nella meta' destra */
         return cercaDicot(v, x, c+1, right);
      else if (x < v[c])
         /* cerca nella meta' destra */
         return cercaDicot(v, x, left, c-1);
      else
         /* trovato */
         return c;
   }
}
```

```
/* ================================================================ */
/*   7                                                              */
/* ================================================================ */

/* TAIL */

#include <stdio.h>
#include <stdlib.h>
#define LEN 10

#define swap(A,B) do{ static int t; t=A; A=B; B=t; }while(0)

void selectionSort(int v[], int len);

int main()
{
   int vett[LEN], i;

   for (i=0; i<LEN; i++)
   {
      printf("Inserire il %do valore: ", i+1);
      scanf("%d", &vett[i]);
   }

   printf("VALORI INTRODOTTI\n");
   for (i=0; i<LEN; i++)
      printf("%d ", vett[i]);

   selectionSort(vett, LEN);

   printf("VALORI RIORDINATI\n");
   for (i=0; i<LEN; i++)
      printf("%d ", vett[i]);

   return EXIT_SUCCESS;
}

void selectionSort(int v[], int len)
{
   int i, minIndex;

   if (len > 1)
   {
      minIndex = 0;
      for (i=1; i<len; i++) /* cerca il minimo */
         if (v[i] < v[minIndex])
            minIndex = i;

      if (v[minIndex] < v[0]) /* e lo mette al primo posto */
         swap(v[minIndex], v[0]);

      selectionSort(v+1, len-1);
   }
  return;    /* caso di base */
}
```